DESARROLLO MÓVIL INTEGRAL

Ingeniería en Desarrollo y Gestión de Software

Manual de Flutter

Instalación y configuración del entorno de desarrollo para aplicaciones móviles multiplataforma

ALUMNO
Brian Jesús Mendoza Márquez

DOCENTE
M.T.I. Marco A. Ramírez Hernández

MATRÍCULA · GRADO Y GRUPO
230308 · 9A

ASIGNATURA
Desarrollo Móvil Integral

Periodo: septiembre – diciembre de 2026
19 de octubre de 2026

Contenido

1. 01. Objetivo y uso del manual
2. 03. Control de versiones
3. 04. Introducción
4. 05. Objetivos
5. 06. Arquitectura del entorno
6. 07. Requisitos de hardware
7. 08. Software necesario
8. 09. Configuración del sistema
9. 10. Visual Studio Code
10. 11. Git
11. 12. GitHub
12. 13. Flutter y Dart
13. 14. Android Studio y Android SDK
14. 15. Emulador Android y dispositivo físico
15. 16. Primer proyecto Flutter
16. 17. Fundamentos de HTTP y REST
17. 18. Postman
18. 19. Alternativa A: Python y FastAPI
19. 20. Swagger y OpenAPI
20. 21. Alternativa B: Node.js y Express
21. 22. PostgreSQL
22. 23. MongoDB
23. 24. PostgreSQL frente a MongoDB
24. 25. Keycloak
25. 26. Flujo de autenticación
26. 27. Docker
27. 28. Dockerfile
28. 29. Docker Compose
29. 30. Variables de entorno y secretos
30. 31. Integración Flutter y API
31. 32. Integración Flutter y Keycloak
32. 33. Plan de pruebas
33. 34. Estructura del repositorio
34. 35. Flujo de ramas Git
35. 36. Seguridad
36. 37. Resolución de problemas
37. 38. Evidencias de aprendizaje
38. 39. Actividad final: gestión de tareas
39. 40. Checklist final
40. 41. Glosario

01. Objetivo y uso del manual

Al finalizar, el estudiante podrá preparar, explicar y verificar un entorno que conecta una aplicación Flutter con una API autenticada, PostgreSQL y MongoDB mediante Docker Compose. El trabajo incluye instalar herramientas, comprender su función y demostrar el flujo completo con evidencias propias.

Cómo estudiar

1. Completa los capítulos 02–18 para preparar la computadora y comprender HTTP.
2. Elige el capítulo 19 (Python/FastAPI) o 21 (Node.js/Express). Lee el 20 en ambos casos.
3. Estudia los datos y la autenticación en 22–26.
4. Monta los servicios siguiendo 27–30.
5. Integra Flutter, prueba y entrega siguiendo 31–41.

Las herramientas se presentan con concepto, problema resuelto, ubicación arquitectónica, instalación, configuración, práctica, verificación y evidencia. Los capítulos conceptuales complementan las prácticas; no requieren instalar otro programa.

Convenciones: los comandos PowerShell parten de la carpeta indicada. `ID_DEL_DISPOSITIVO`, `TU_USUARIO` y los campos de portada se sustituyen por datos propios. Los archivos de código del proyecto sí están completos. Los resultados esperados no implican que una prueba se haya ejecutado.

La ruta principal prueba Android. Flutter permite otras plataformas, pero iOS exige macOS y Xcode para compilar; el login de este proyecto está configurado para Android. Usar Chrome con `flutter run` no sustituye esta prueba de autenticación móvil.

Entrega: repositorio, manual con portada, evidencias E01–E24 y demostración de tareas autenticadas. Consulta el índice en cualquier momento.

03. Control de versiones

Versión	Fecha	Autor	Cambio realizado
1.0	2026-10-16	Equipo del proyecto académico	Manual por capítulos, dos backends alternativos y aplicación de tareas.

Al modificar un procedimiento, registra fecha, autor y explicación verificable. Diferencia la versión del documento de las versiones de Flutter, Python o una imagen Docker.

Antes de entregar, registra en E24 las versiones reales y el commit evaluado. Las etiquetas de imágenes con versión mayor permiten recibir correcciones de esa familia; para repetir exactamente una ejecución conserva también sus digests y los archivos de bloqueo de dependencias.

04. Introducción

El desarrollo móvil multiplataforma busca compartir una parte importante del código entre sistemas. Una aplicación nativa suele usar las herramientas propias de cada plataforma, por ejemplo Kotlin para Android y Swift para iOS. Una solución multiplataforma reduce duplicación, aunque todavía debe configurar permisos, compilación y funciones particulares de cada sistema.

Flutter construye interfaces mediante widgets y utiliza Dart. Un widget representa una parte de la pantalla o su comportamiento: un texto, un botón, una lista o una composición de ellos. Compartir código no garantiza que todos los paquetes funcionen en todas las plataformas.

En una arquitectura **cliente-servidor**, Flutter es el cliente: muestra datos y solicita operaciones. El **backend** procesa esas solicitudes, aplica reglas y consulta bases de datos. Una **API REST** organiza esas operaciones alrededor de recursos y métodos HTTP, como consultar tareas con GET o crearlas con POST.

Una **base de datos** conserva información más allá del cierre de la aplicación. PostgreSQL organiza datos relacionados en tablas; MongoDB guarda documentos. En este proyecto las tareas van a PostgreSQL y los eventos de actividad a MongoDB.

La **autenticación** comprueba quién es el usuario; la **autorización** decide qué puede hacer. Keycloak gestiona el inicio de sesión. La API valida el token recibido antes de acceder a los datos y comprueba además la propiedad de cada tarea.

La **contenerización** empaqueta procesos y dependencias para ejecutarlos de forma consistente. Docker ejecuta contenedores y Compose describe cómo colaborar entre ellos. El **control de versiones** conserva el historial de cambios; Git lo administra localmente y GitHub facilita compartirlo y revisarlo.

Ejemplo: Alex inicia sesión, crea una tarea y ve su nueva lista. Flutter gestiona la pantalla; Keycloak identifica a Alex; la API valida la solicitud; PostgreSQL conserva la tarea; MongoDB registra la actividad.

05. Objetivos

Objetivo general

Instalar, configurar y validar un entorno reproducible para desarrollar una aplicación móvil multiplataforma con servicios REST, persistencia, autenticación y control de versiones.

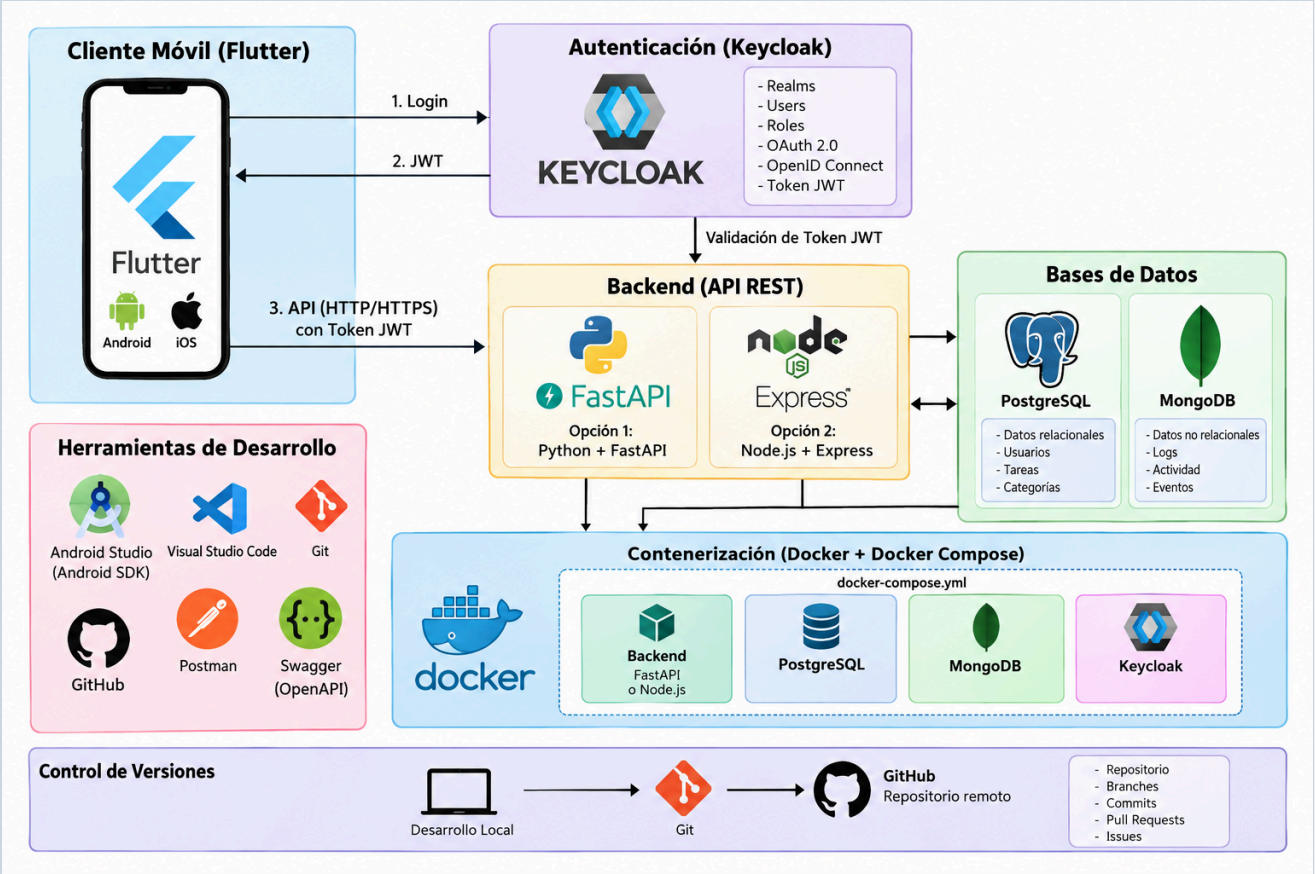
Objetivos específicos

- Configurar Flutter, Dart y Android SDK; interpretar `flutter doctor`.
- Crear y ejecutar una aplicación Android con Flutter.
- Consumir APIs y manejar carga, errores, JSON y tiempos de espera.
- Crear una API REST con FastAPI **o** Express.
- Utilizar PostgreSQL para relaciones y MongoDB para documentos de actividad.
- Implementar login con Keycloak y permisos mediante roles y propietario.
- Probar solicitudes con Postman y explorar el contrato OpenAPI mediante Swagger.
- Crear imágenes Docker y administrar servicios con Docker Compose.
- Utilizar Git para registrar cambios y GitHub para colaborar con ramas y Pull Requests.
- Presentar evidencias reproducibles sin divulgar contraseñas ni tokens.

El criterio de logro es funcional: cada estudiante debe poder explicar y demostrar el recorrido desde un toque en Flutter hasta la persistencia y la respuesta.

06. Arquitectura del entorno

```
flowchart TD
  Flutter[Flutter / Dart] -->|HTTP / HTTPS: API REST| Backend
  subgraph Compose[Docker Compose: entorno backend]
    Backend[Backend: FastAPI O Express] --> PG[(PostgreSQL: users, tasks, categories)]
    Backend --> MG[(MongoDB: logs, activity)]
    KC[Keycloak] --> KDB[(PostgreSQL separado: identidad)]
  end
  Flutter -->|Authorization Code + PKCE| KC
  KC -->|Access token JWT| Flutter
  Flutter -->|Authorization: Bearer JWT| Backend
  Backend -->|Claves públicas JWKS| KC
  Git[Git local] --> GitHub[GitHub remoto / Pull Requests]
  Git -. versiona código y configuración .-> Compose
  Git -. versiona .-> Flutter
```



Vista general de la arquitectura DevOps: Flutter, backend, PostgreSQL, MongoDB, Keycloak, Docker y Git.

FastAPI y Express implementan el mismo contrato; elige una alternativa. Keycloak mantiene su propia base de identidad y credenciales, separada de las tablas de aplicación. No se conectan PostgreSQL o MongoDB directamente desde Flutter.

En el laboratorio HTTP facilita la configuración local; en producción se utiliza HTTPS. El backend usa nombres de servicio (`postgres`, `mongo`, `keycloak`) dentro de Docker. Un teléfono necesita una dirección accesible desde su red o el puente USB descrito en el capítulo 32.

La escritura de tareas y la auditoría en MongoDB no forman una transacción distribuida. Si falla MongoDB después de guardar una tarea, la respuesta indica `audit_recorded: false`; el capítulo 39 explica esta limitación deliberada.

07. Requisitos de hardware

Estas cifras son un presupuesto académico orientativo para este laboratorio, no sustituyen los requisitos oficiales de cada herramienta.

Configuración mínima práctica

Recurso	Propuesta
CPU	64 bits, compatible con virtualización
RAM	8 GB, usando celular físico y evitando cargas simultáneas
Disco	SSD; al menos 30 GB libres para comenzar, ampliables
Virtualización	Habilitada en firmware y disponible para WSL 2
Red	Internet para SDK, imágenes y dependencias

Configuración recomendada

Recurso	Propuesta
CPU	Core i5 / Ryzen 5 o superior, varios núcleos
RAM	16 GB; 32 GB da mayor margen para emulador y servicios
Disco	SSD, 40–50 GB libres inicialmente
Virtualización	Activa; WSL 2 operativo
Dispositivo	Android físico por USB o AVD

El emulador ejecuta un sistema Android completo. Docker Desktop necesita recursos para su entorno Linux; Keycloak ejecuta Java; ambas bases mantienen cachés. Además, Gradle compila Android y el editor indexa código. Con 8 GB usa un teléfono, cierra aplicaciones pesadas y mide consumo en el Administrador de tareas.

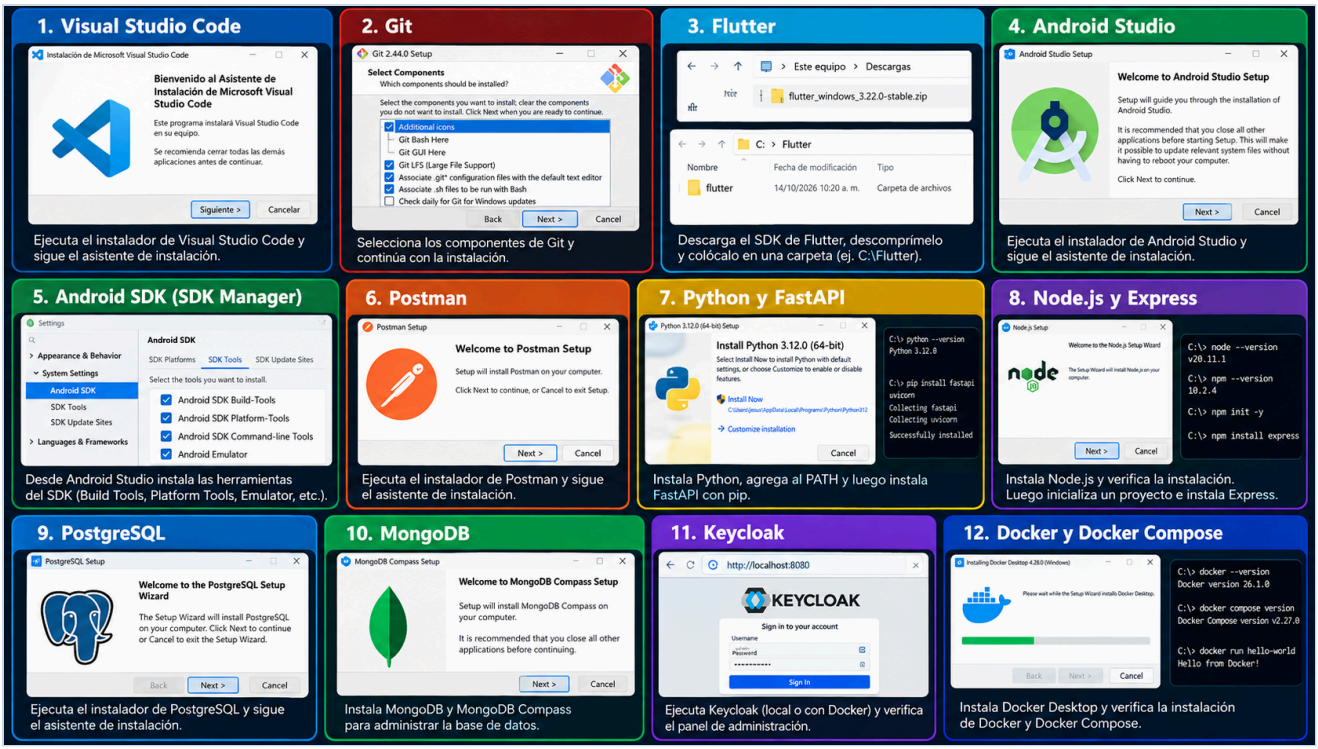
Si las versiones instaladas requieren más memoria o un Windows más reciente, prevalecen sus requisitos oficiales: Android Studio y Docker Desktop para Windows.

Evidencia E15: procesador, memoria, espacio disponible y virtualización; oculta identificadores personales del equipo.

08. Software necesario

Herramienta	Función	Obligatoria/Opcional
VS Code	Edición y depuración	Recomendado; otro IDE es posible
Git	Historial local	Obligatoria
GitHub	Repositorio remoto y revisión	Obligatoria para la entrega
Flutter	SDK de aplicación móvil	Obligatoria
Dart	Lenguaje, incluido con Flutter	Obligatoria; sin instalación separada
Android Studio	SDK Manager y emulador	Obligatoria en esta ruta
Android SDK	Compilación y conexión Android	Obligatoria
Postman	Pruebas de API	Obligatoria
Docker Desktop	Motor Linux y Compose en Windows	Obligatoria en esta ruta
Docker Compose	Servicios coordinados	Obligatoria; incluida en Desktop
PostgreSQL	Datos relacionales	Obligatoria, instalada como contenedor
MongoDB	Actividad y registros	Obligatoria, instalada como contenedor
Python	Runtime de FastAPI	Alternativa A; local opcional con Docker
FastAPI	API Python	Alternativa A
Node.js y npm	Runtime y dependencias JavaScript	Alternativa B; local opcional con Docker
Express	API JavaScript	Alternativa B
Keycloak	Identidad y login	Obligatoria, como contenedor
Swagger UI / OpenAPI	Interfaz y contrato de API	Incluidos con los backends
MongoDB Compass	Exploración gráfica	Opcional
Cliente PostgreSQL	Consultas SQL	`psql` incluido en el contenedor

No instales servidores nativos de PostgreSQL o MongoDB si seguirás la ruta Docker: podrían ocupar los mismos puertos. Sí puedes utilizar sus clientes gráficos desde Windows.



Resumen visual de las herramientas de desarrollo que se instalan en esta ruta.

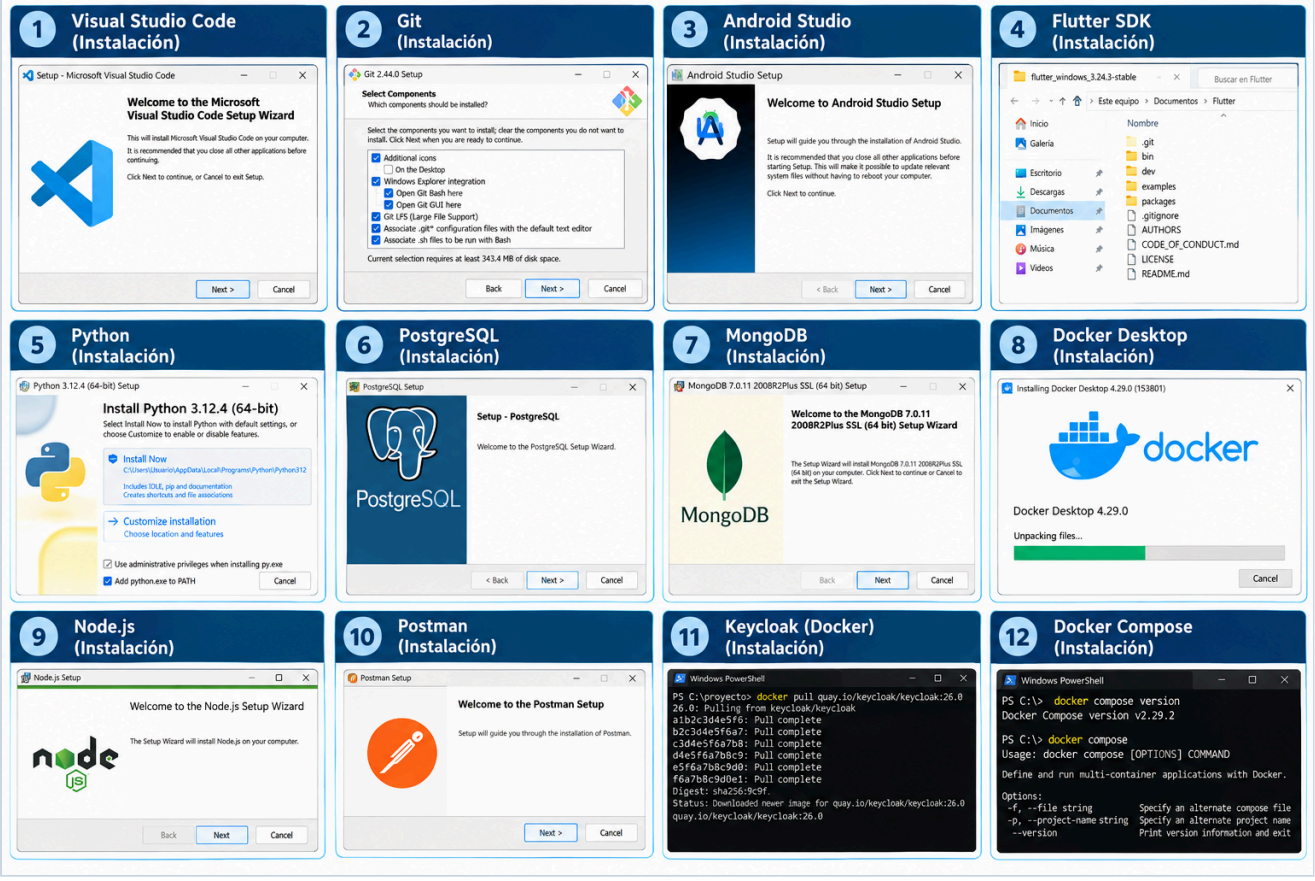
09. Configuración del sistema

Una variable de entorno es un valor que un proceso recibe del sistema, como la ruta de un SDK. `PATH` enumera directorios donde la terminal busca ejecutables. `JAVA_HOME` identifica un JDK; `ANDROID_HOME`, el Android SDK.

PowerShell es un intérprete de comandos. La terminal es la ventana que lo aloja. Una variable `$env:NOMBRE` corresponde al entorno del proceso actual.

Preparación en Windows

1. Actualiza Windows y confirma compatibilidad con Docker Desktop.
2. En Administrador de tareas → Rendimiento → CPU revisa **Virtualización**. Si está desactivada, consulta el procedimiento del fabricante para habilitar VT-x/AMD-V en UEFI.
3. Instala WSL 2 siguiendo el asistente de Docker Desktop. Reinicia cuando se solicite.
4. Abre PowerShell normal para trabajar; eleva permisos solo cuando un instalador del sistema lo requiera.



Secuencia visual de la preparación e instalación en Windows.

Ejemplo de configuración de la sesión después de instalar los SDK:

```
$env:ANDROID_HOME = "$env:LOCALAPPDATA\Android\Sdk"
```

```
$env:JAVA_HOME = 'C:\Program Files\Android\Android Studio\jbr'
```

```
$env:Path = "$env:USERPROFILE\develop\flutter\bin;$env:ANDROID_HOME\platform-tools;$env:Path"
```

Comprueba las rutas antes de usarlas. Para persistirlas, abre **Variables de entorno de usuario** y añade entradas; no reemplaces todo el PATH. Abre una terminal nueva después.

```
where.exe flutter
```

```
flutter doctor -v
```

Flutter puede seleccionar el JDK de Android Studio aunque exista otro `JAVA_HOME`; el diagnóstico informa el Java realmente usado. Si hay incompatibilidad de Gradle, revisa esa ruta antes de instalar versiones al azar.

E15: registra rutas y diagnóstico. Error frecuente: cambiar una variable y continuar usando una terminal que aún conserva el valor anterior.

Fuentes: PATH de Flutter, variables Android, WSL.

10. Visual Studio Code

¿Qué es y para qué sirve?

Editor extensible que reúne archivos, terminal y depurador. Resuelve el trabajo disperso entre ventanas, pero no instala por sí mismo los SDK del proyecto.

Papel dentro de la arquitectura y requisitos

Herramienta de desarrollo, no servicio en producción. Requiere Windows compatible e Internet para extensiones.

Instalación y configuración

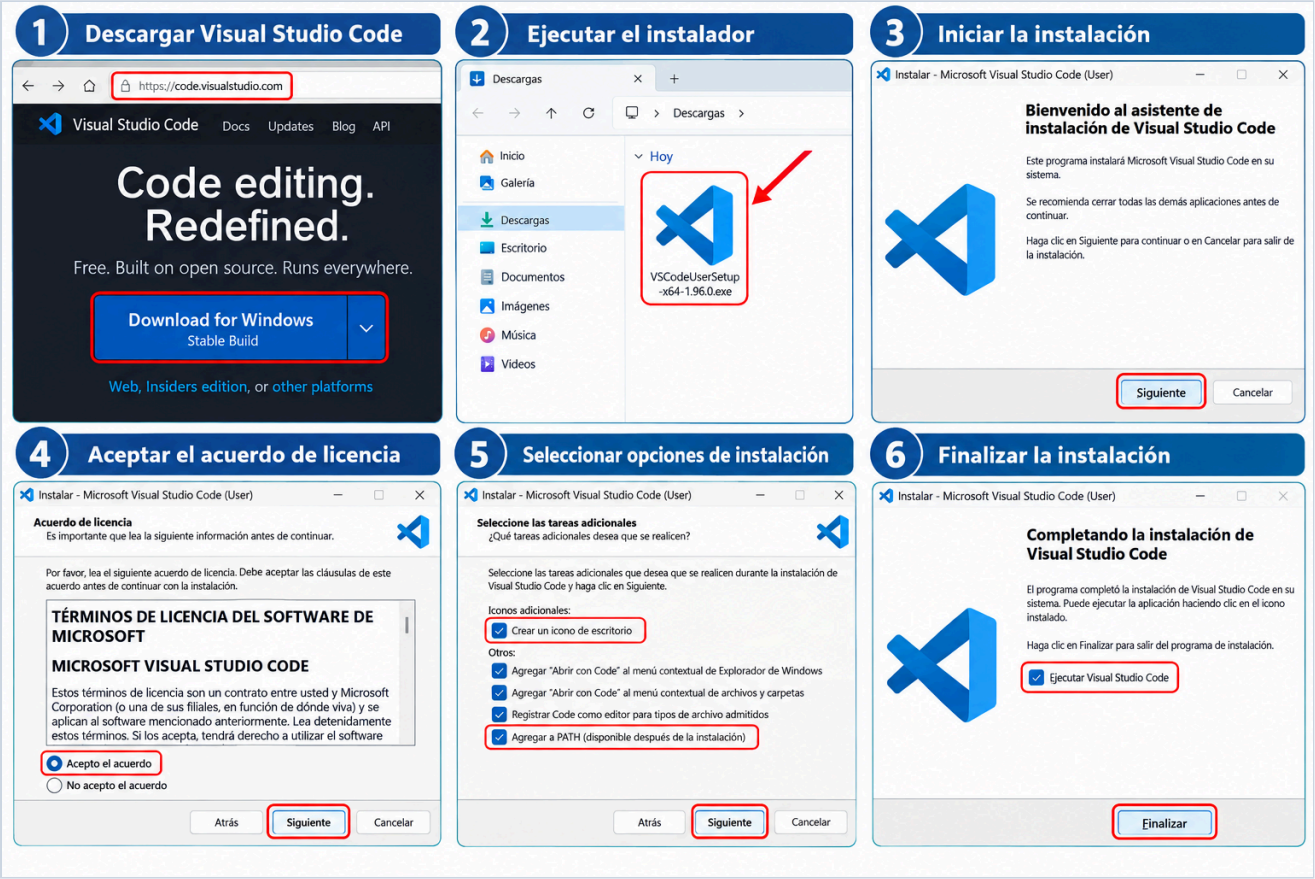
Instala desde VS Code, abre `proyecto-movil` con **File** → **Open Folder** y usa **Terminal** → **New Terminal**. Selecciona PowerShell. En extensiones instala según el módulo:

Descarga e instalación de Visual Studio Code.

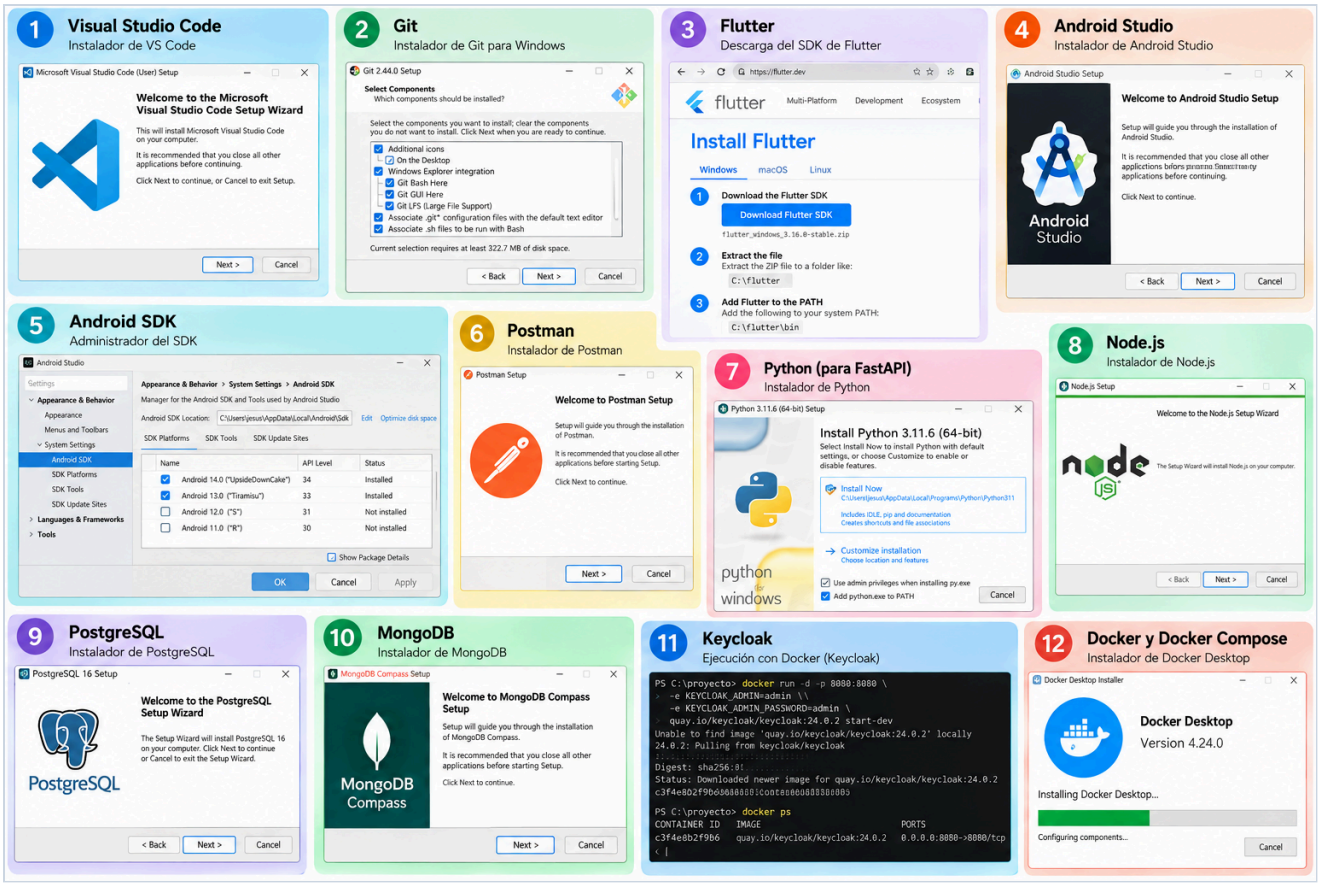
Extensión	Publicador a comprobar	Uso
Flutter y Dart	Dart Code	Análisis, depuración y dispositivos

Extensión	Publicador a comprobar	Uso
Python	Microsoft	Intérprete y pruebas Python
Docker / Container Tools	Microsoft	Imágenes y contenedores
GitHub Pull Requests	GitHub	Revisar PR e Issues
PostgreSQL	Microsoft u otro publicador verificado	Consultas SQL; opcional
MongoDB for VS Code	MongoDB	Documentos y consultas

Los nombres del catálogo pueden cambiar: revisa publicador y permisos antes de instalar. Elige `.venv` como intérprete Python y la carpeta del Flutter SDK cuando el editor lo solicite.

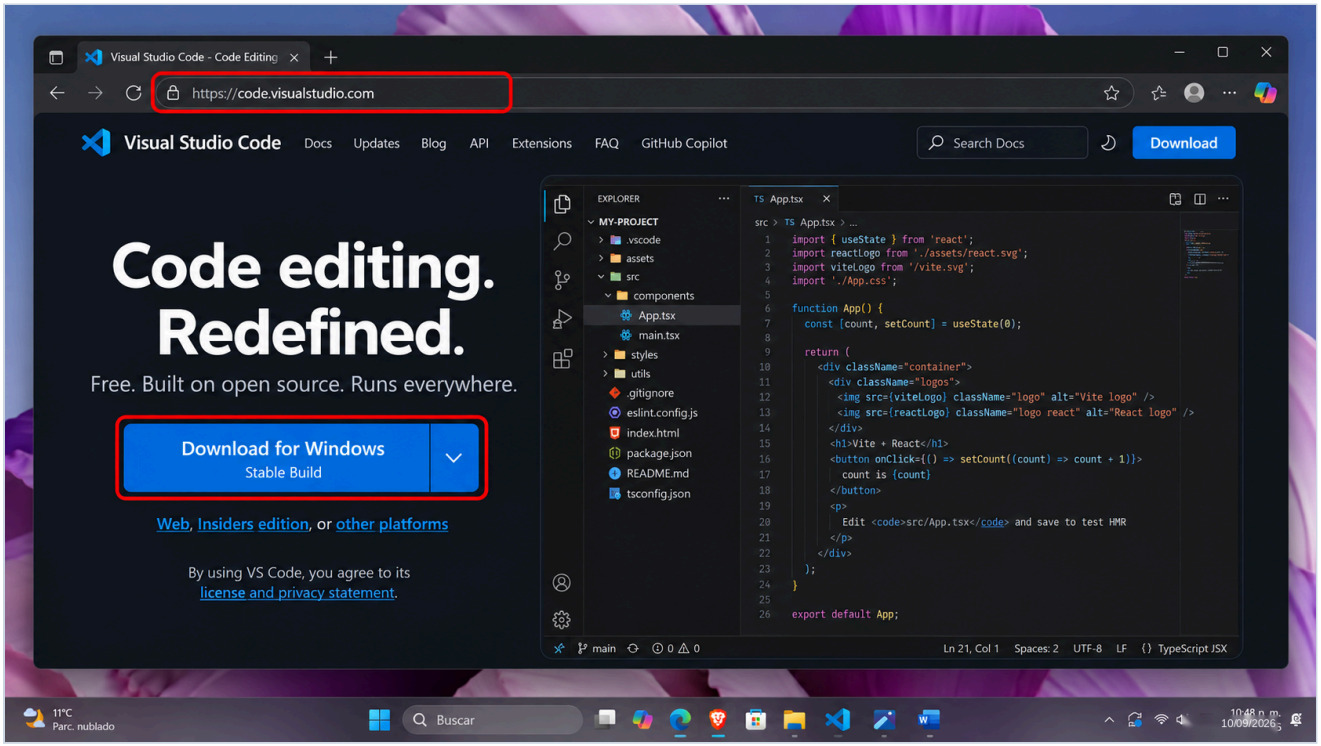


Visual Studio Code abierto en Windows con el proyecto cargado.

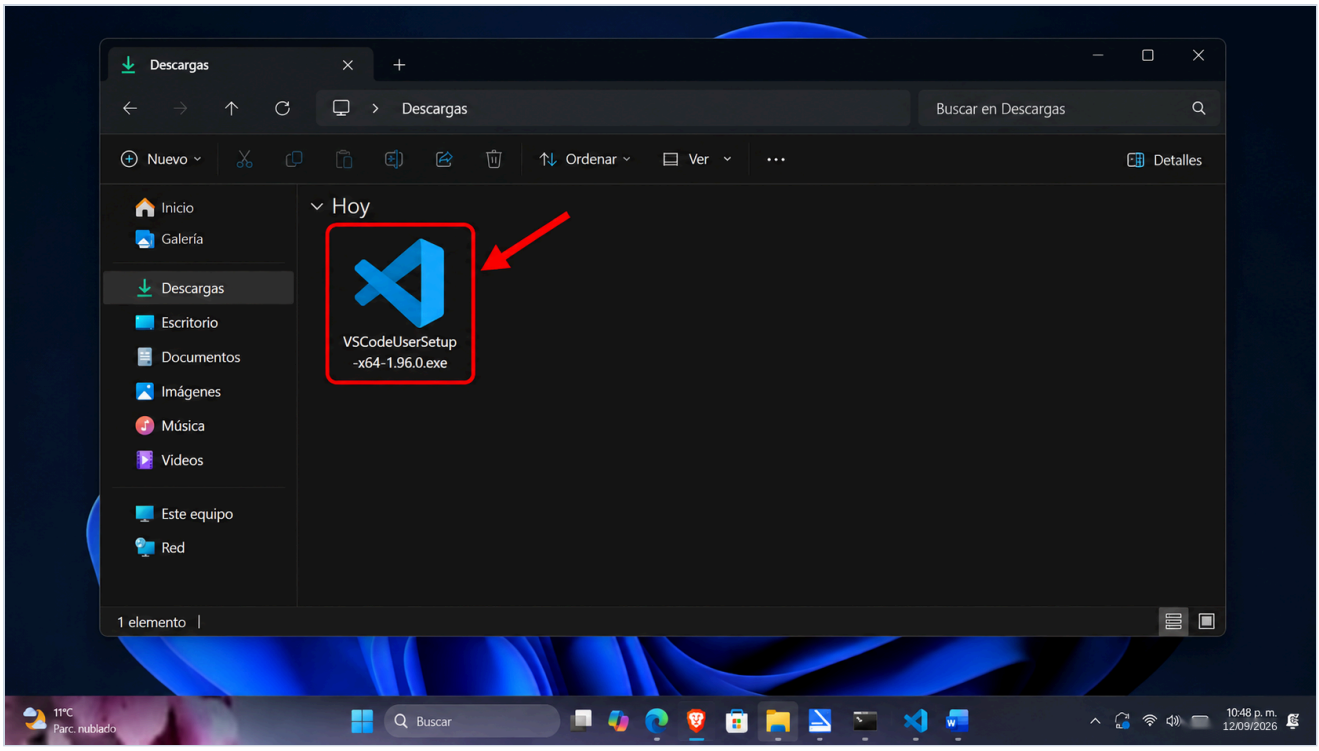


Zona de archivos, editor, terminal integrada y panel de extensiones.

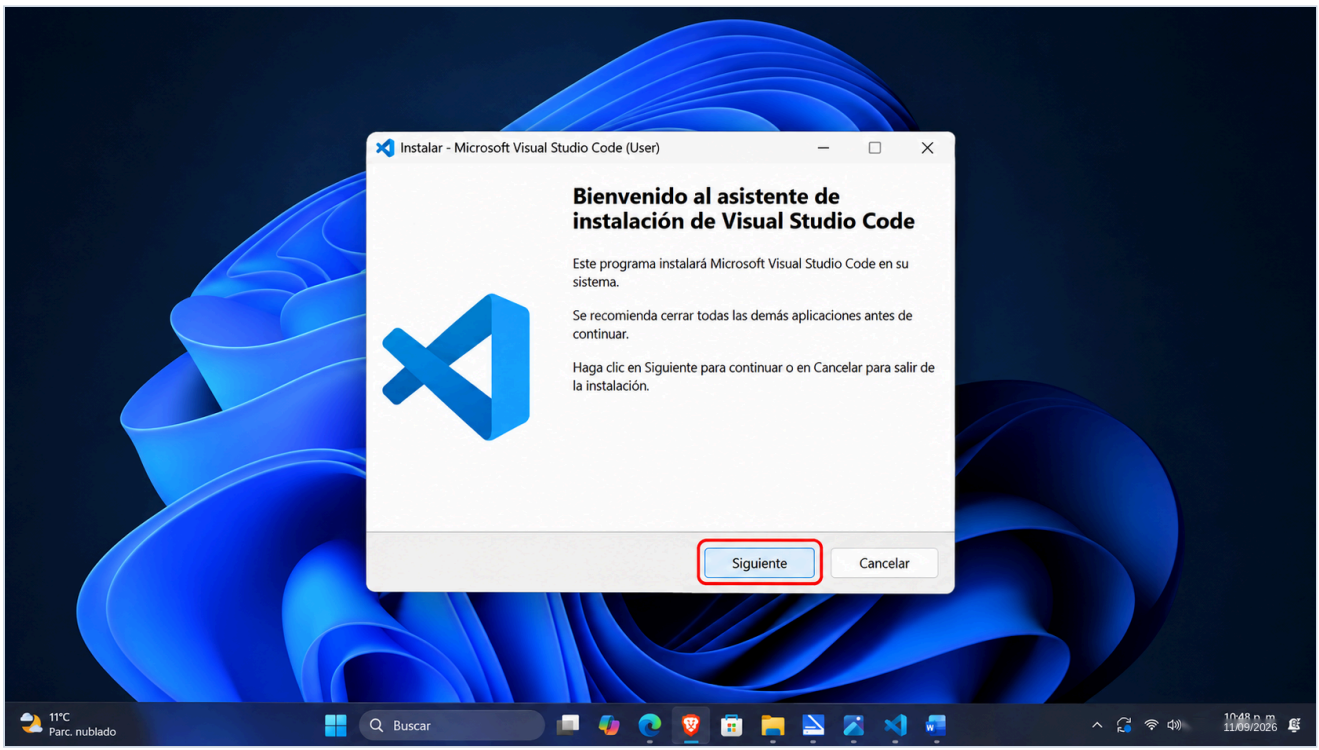
Guía visual de instalación



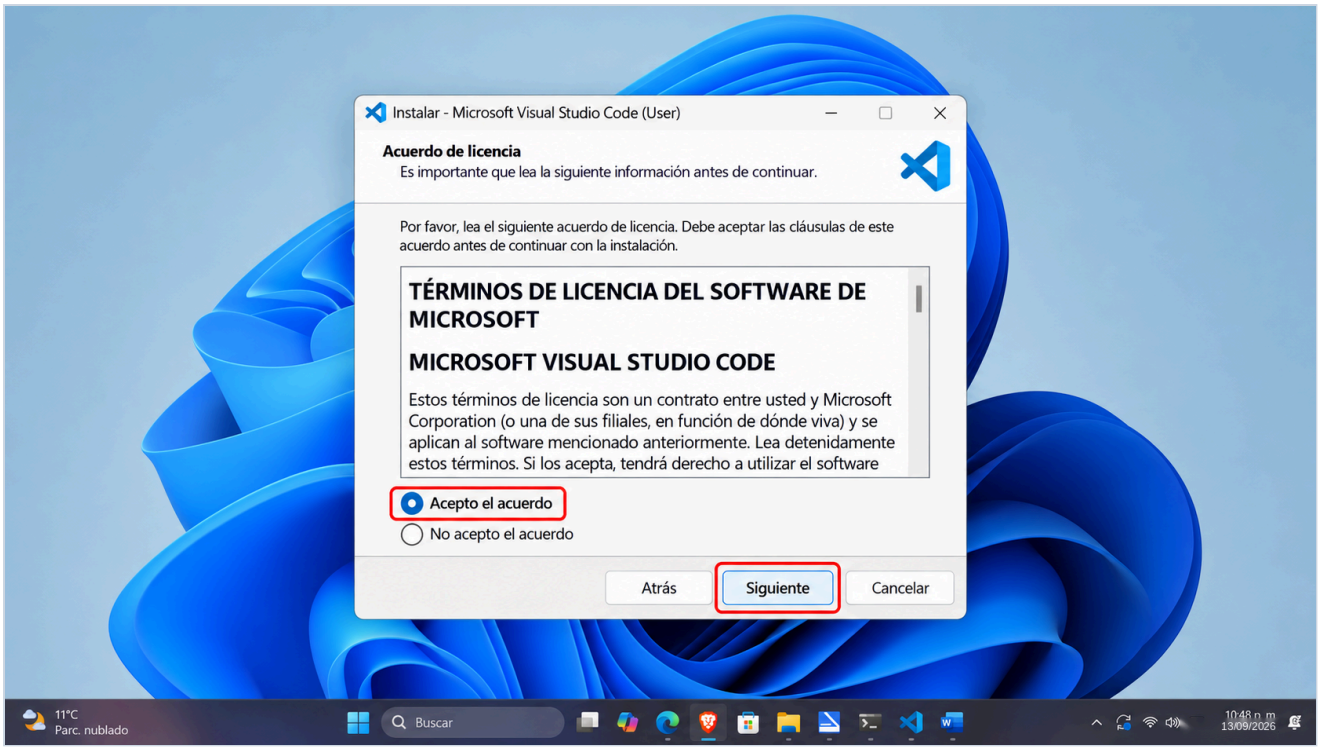
Paso 1. Página de descarga de Visual Studio Code para Windows.



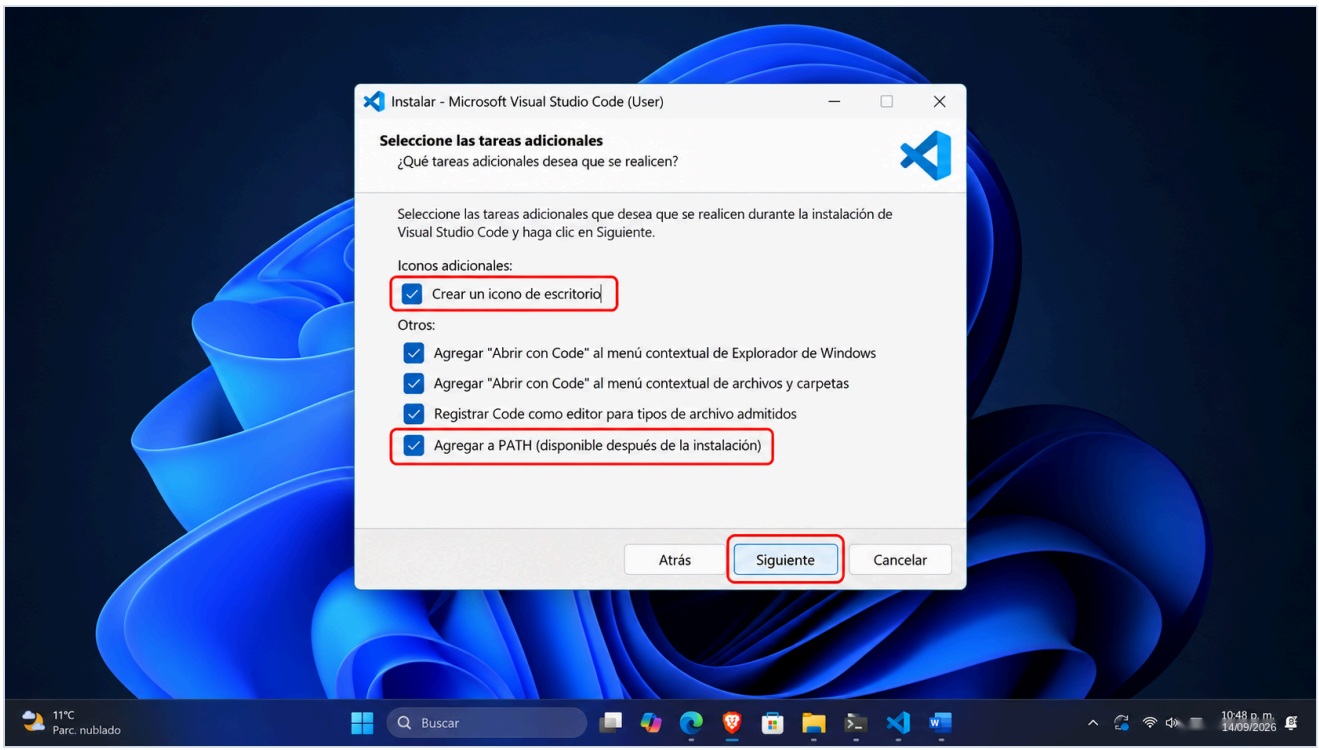
Paso 2. Localiza el instalador descargado en la carpeta Descargas.



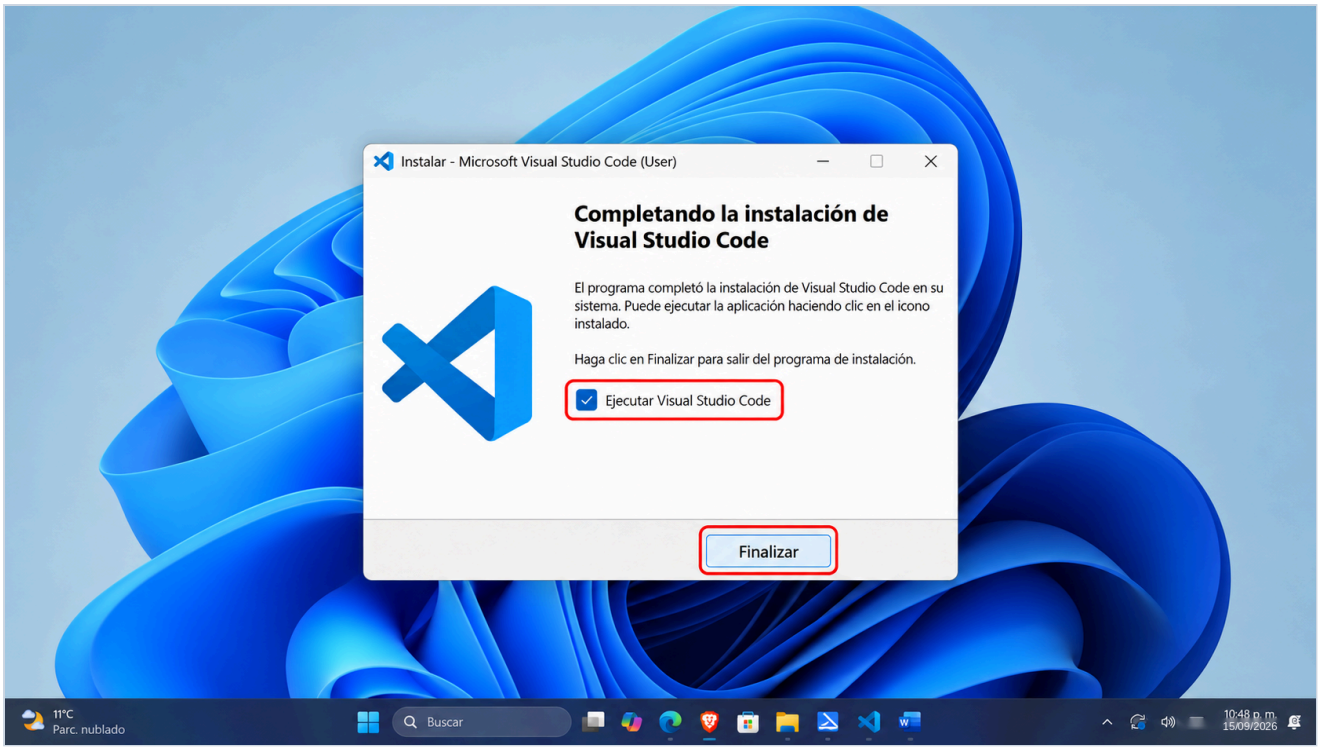
Paso 3. Abre el asistente de instalación y pulsa Siguiente.



Paso 4. Revisa y acepta el acuerdo de licencia para continuar.



Paso 5. Selecciona las tareas adicionales, incluido agregar VS Code al PATH.



Paso 6. Finaliza la instalación y abre Visual Studio Code.

Comandos principales y ejemplo práctico

Desde `proyecto-movil`:

```
code .
```

Abre un archivo Markdown y su vista previa con `Ctrl + Shift + V`. Para Flutter usa **Flutter: Select Device** en la paleta y `F5`.

Verificación, errores comunes y soluciones

Si `code` no se reconoce, abre la carpeta desde el menú y revisa la opción PATH del instalador. Si faltan sugerencias de Dart, abre la carpeta con `pubspec.yaml` y comprueba la extensión.

Evidencia y conclusión

E16: editor, extensiones e intérprete seleccionados. Concluye qué SDK utiliza cada extensión. Referencia Flutter en VS Code.

11. Git

¿Qué es y para qué sirve?

Sistema distribuido de control de versiones. Resuelve la pérdida del historial y permite comparar cambios, trabajar en paralelo y recuperar versiones.

Concepto	Significado
Repositorio	Archivos y su historial
Commit	Registro de cambios con mensaje y autor
Branch	Línea de trabajo
Merge	Integración de líneas de trabajo
Clone	Copia de un repositorio y su historial
Push	Envío de commits al remoto
Pull	Obtención e integración de cambios remotos
Remote	Nombre y dirección de otro repositorio

Papel dentro de la arquitectura y requisitos

Herramienta externa al sistema ejecutándose. Requiere acceso de escritura a la carpeta; Internet solo para colaborar con remotos.

Instalación y configuración

Instala Git para Windows, abre una terminal nueva y verifica:

```
git --version
```

```
git config --global user.name "Nombre"
```

```
git config --global user.email "correo@ejemplo.com"
```

El correo queda en los commits; puedes usar el correo privado proporcionado por GitHub.

Comandos principales y ejemplo práctico

En una carpeta nueva de ejercicio, fuera de un repositorio existente:

```
git init
```

Después de crear un README y `.gitignore`, revisa los archivos antes de agregarlos:

```
git status
```

```
git add .
```

```
git commit -m "Primer commit"
```

```
git branch
```

```
git switch -c feature/manual
```

`git switch main` vuelve a una rama existente; `-c` crea una nueva. Los comandos de remoto se practican en el capítulo siguiente.

Verificación, errores comunes y soluciones

```
git log --oneline -5
```

Si no conoce tu identidad, configura nombre y correo. Si `git add .` incluye secretos, retíralos del área de preparación antes del commit y corrige `.gitignore`. No inicialices otro `.git` dentro de este proyecto: ya pertenece al repositorio de prácticas.

Evidencia y conclusión

E01: versión, estado y un commit. Explica la diferencia entre guardar un archivo y registrar un commit. Referencia de Git.

12. GitHub

¿Qué es y para qué sirve?

Plataforma que aloja repositorios Git y facilita revisión y coordinación. Git funciona localmente; GitHub agrega repositorios remotos, Pull Requests e Issues. Un PR propone integrar cambios; un Issue registra una tarea o problema.

Papel dentro de la arquitectura y requisitos

Herramienta externa de colaboración. Necesitas Git, cuenta GitHub e Internet. Autentica Git mediante el navegador con Git Credential Manager o configura SSH; no uses la contraseña de tu cuenta como contraseña de Git por HTTPS.

Instalación y configuración

No necesita instalar un servidor. Crea tu cuenta en GitHub, activa autenticación en dos pasos y crea un repositorio de práctica con README. Copia su URL desde **Code**.

Comandos principales y ejemplo práctico

Sustituye `TU_USUARIO` por tu cuenta y ejecuta fuera del repositorio actual:

```
git clone https://github.com/TU_USUARIO/proyecto-movil.git
```

```
Set-Location proyecto-movil
```

```
git switch -c feature/documentacion
```

Edita README, guarda y registra:

```
git add README.md
```

```
git commit -m "Documenta instalación inicial"
```

```
git push -u origin feature/documentacion
```

En GitHub, abre **Compare & pull request**, selecciona base `main`, describe cambios y pruebas y solicita revisión. Crea también un Issue con un resultado esperado concreto.

```
git pull --ff-only
```

Verificación, errores comunes y soluciones

Comprueba la rama y el commit en el sitio. Si rechaza el push, revisa permisos, remoto y cambios ajenos; no fuerces la sobrescritura. Para un historial divergente consulta el capítulo 37.

Evidencia y conclusión

E13: URL del repositorio, PR e Issue. Explica qué revisaría un compañero antes de integrar. Guía de PR.

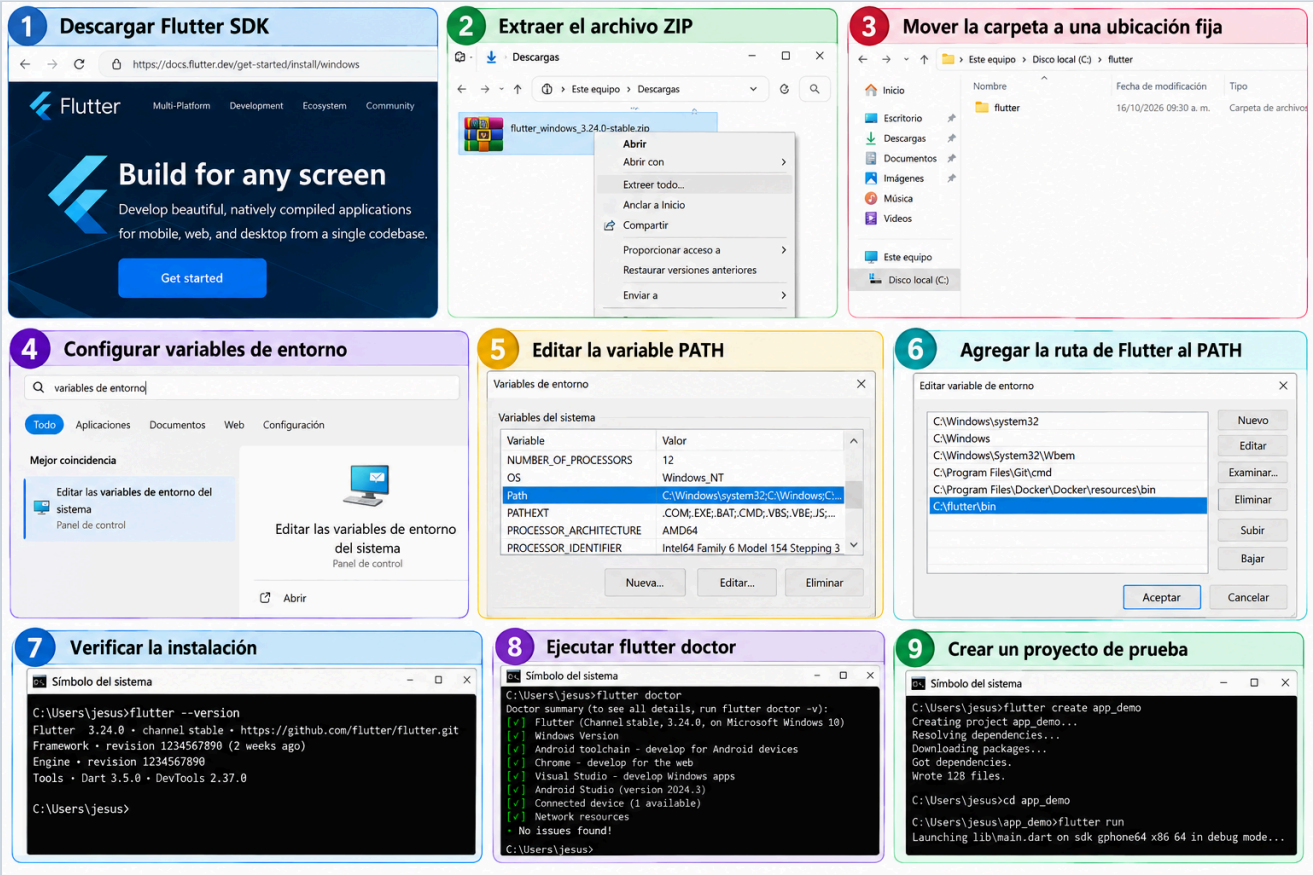
13. Flutter y Dart

Flutter: ¿qué es y para qué sirve?

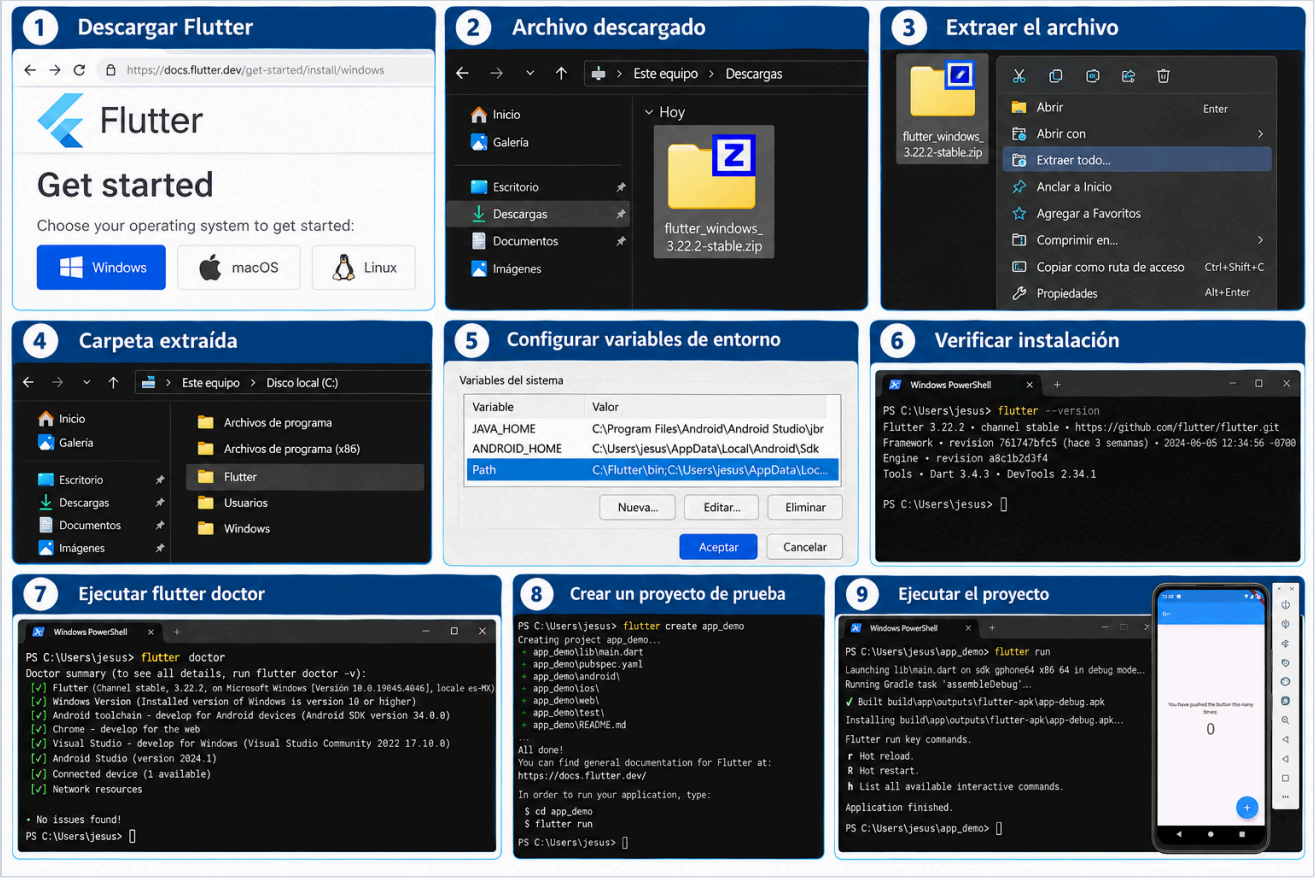
SDK para construir aplicaciones mediante widgets. Resuelve la necesidad de mantener interfaces independientes para cada plataforma, compartiendo código cuando es posible. Es la capa cliente de este proyecto.

Requisitos previos e instalación

Necesitas Git y una carpeta de herramientas con permisos de escritura. Descarga el SDK **stable** para Windows desde la instalación oficial, extrae en `C:\Users\TuUsuario\develop\flutter` y agrega su carpeta `bin` al PATH. Comprueba que exista `bin\flutter.bat`.

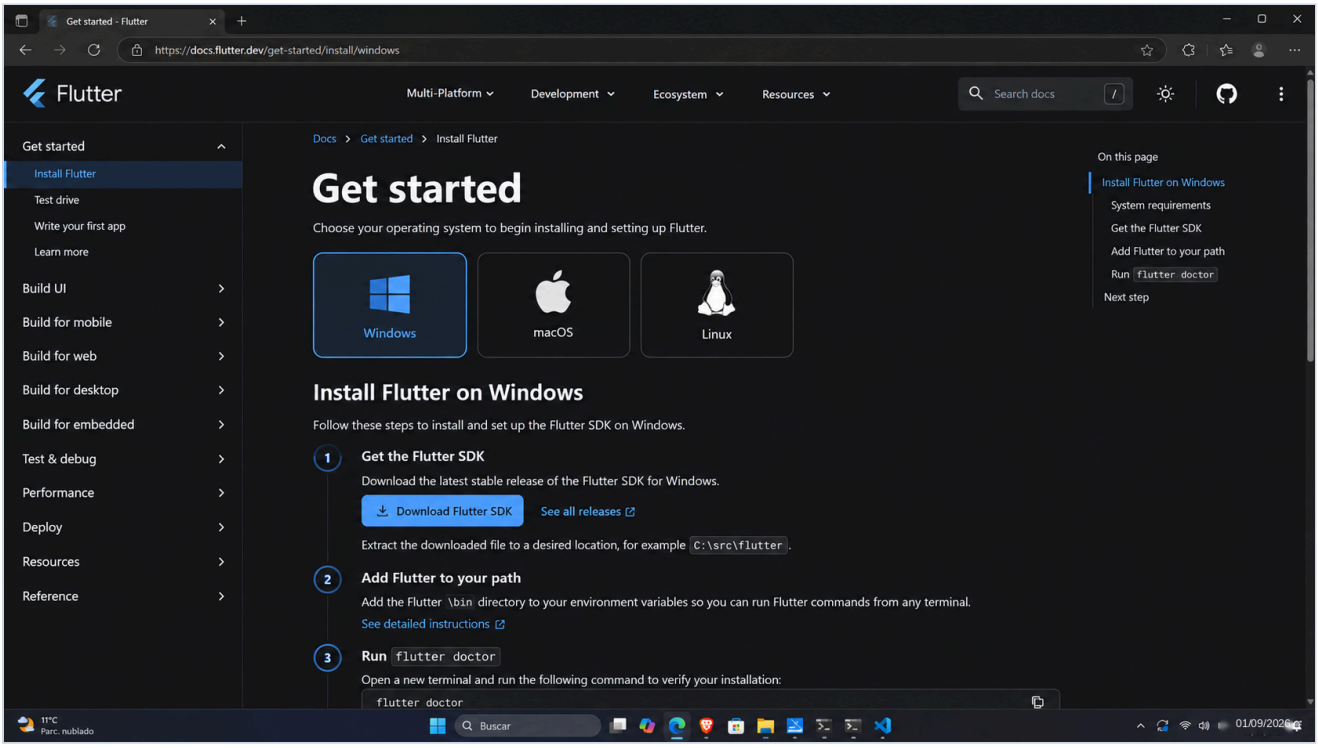


Descarga del SDK stable de Flutter para Windows.

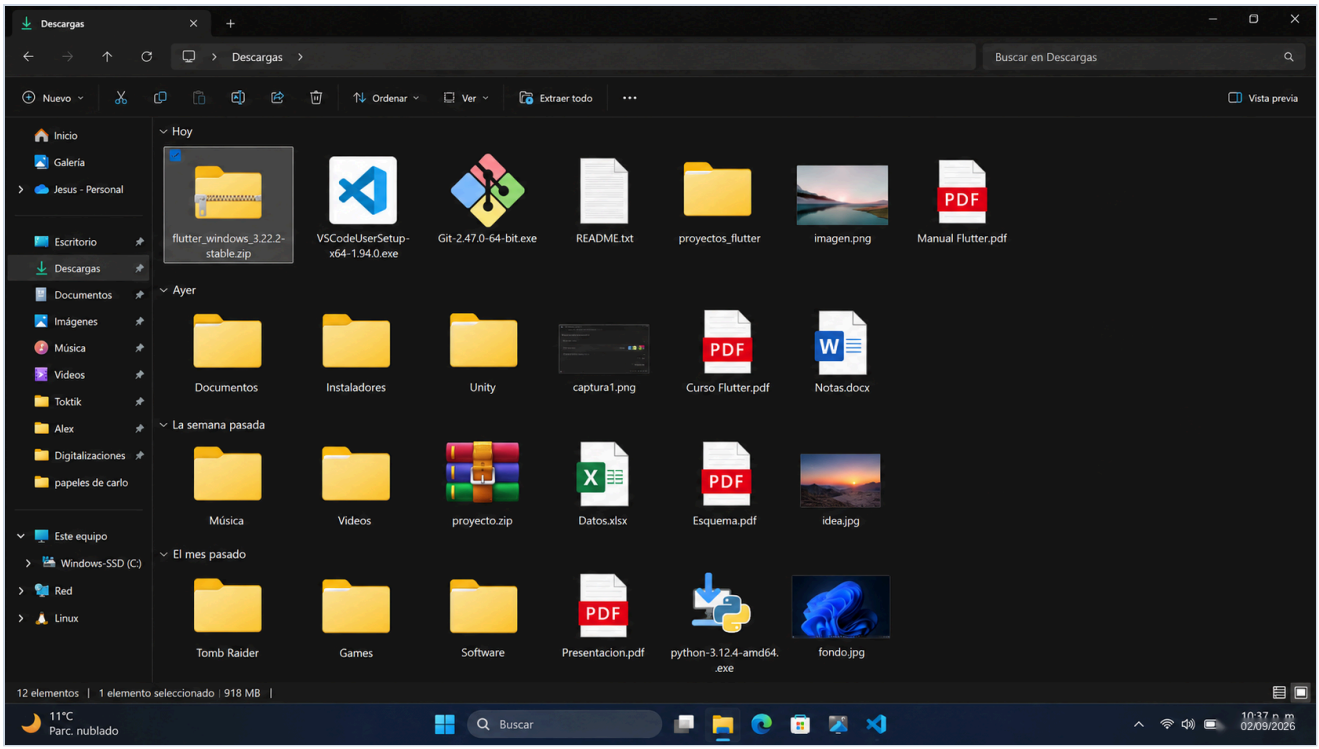


Extracción del SDK y adición de la carpeta `bin` al PATH.

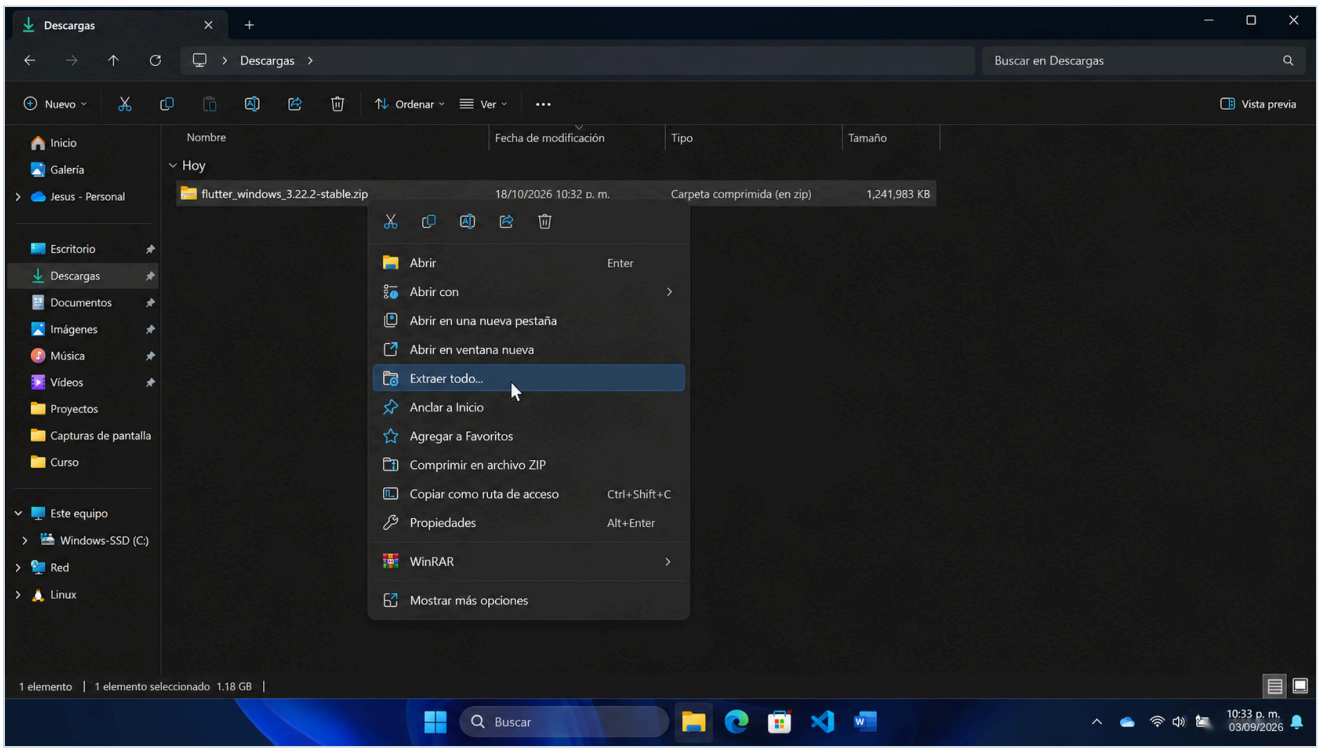
Guía visual de instalación del SDK



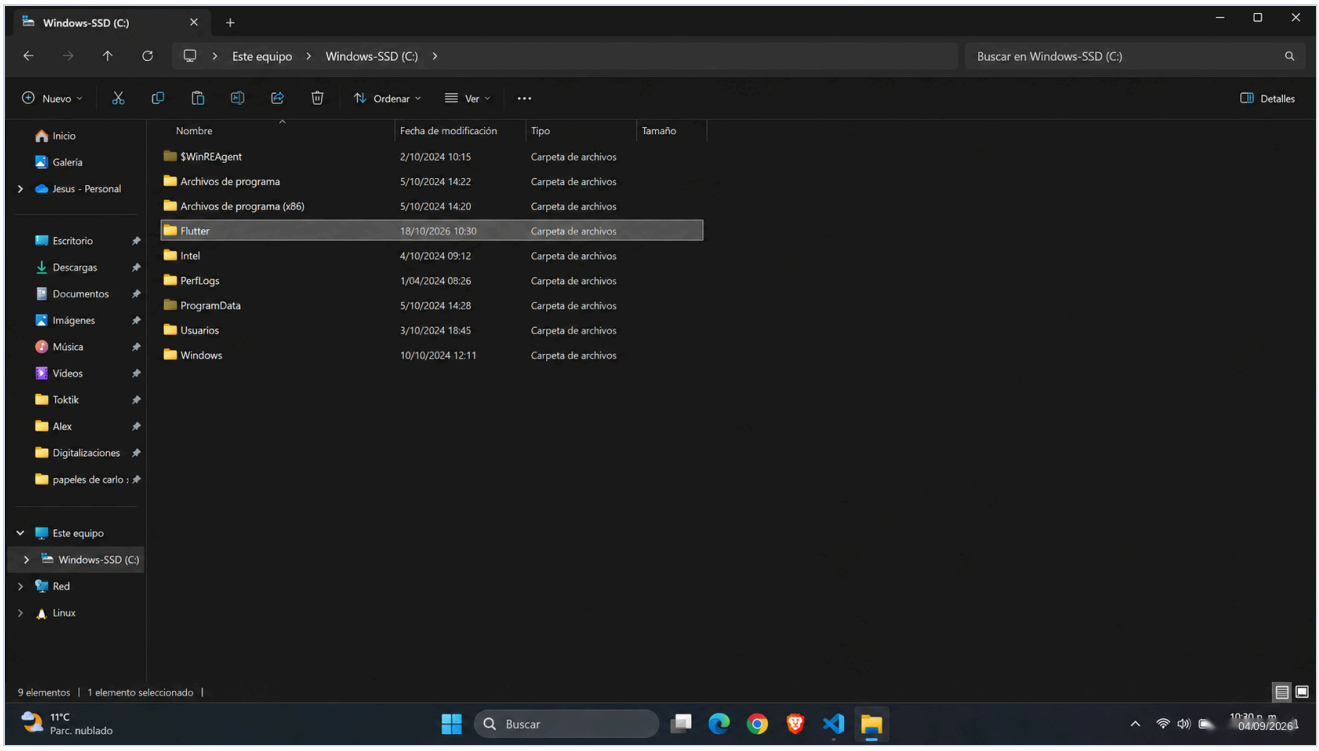
Paso 1. Selecciona Windows en la guía de instalación de Flutter.



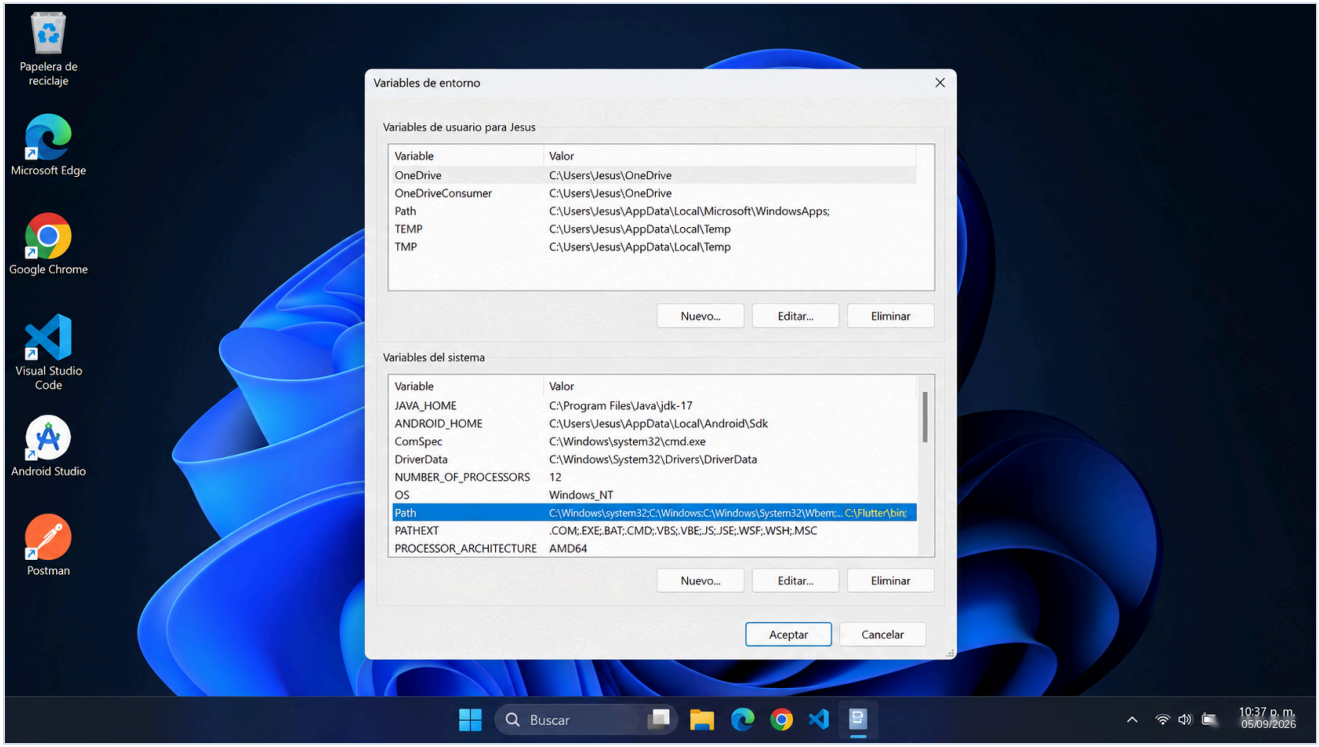
Paso 2. Localiza el archivo del SDK descargado.



Paso 3. Extrae el archivo ZIP del SDK de Flutter.



Paso 4. Ubica la carpeta de Flutter en el disco.



Paso 5. Configura la variable PATH para incluir la carpeta bin del SDK.

Configuración y comandos principales

```
flutter --version
```

Informa Flutter, canal y Dart incluidos.

```
flutter doctor
```

Diagnóstico general.

```
flutter doctor -v
```

Diagnóstico con rutas y versiones.

```
flutter devices
```

Lista dispositivos disponibles.

```
flutter create app_demo
```

Crea el ejemplo en una carpeta nueva. Dentro de ella:

```
flutter pub get
```

Resuelve paquetes definidos en `pubspec.yaml`.

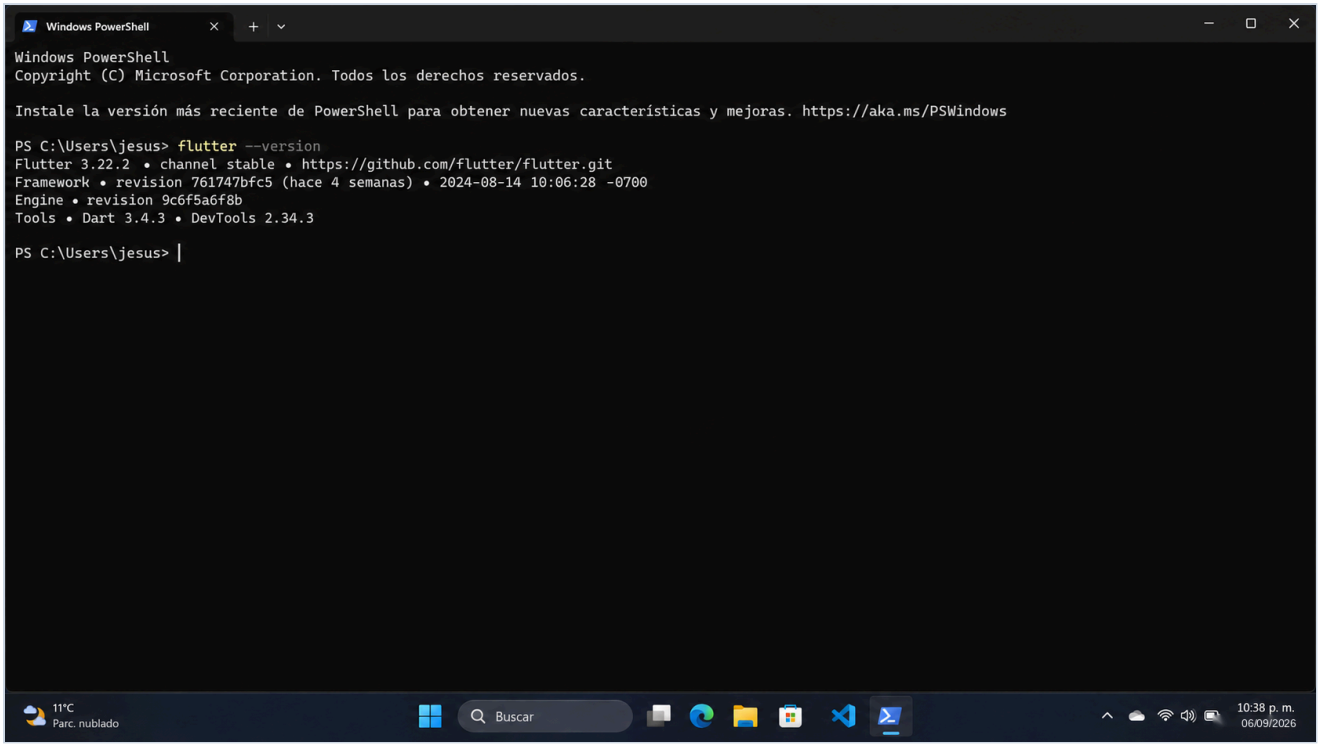
```
flutter run
```

Compila y ejecuta; selecciona dispositivo si lo solicita.

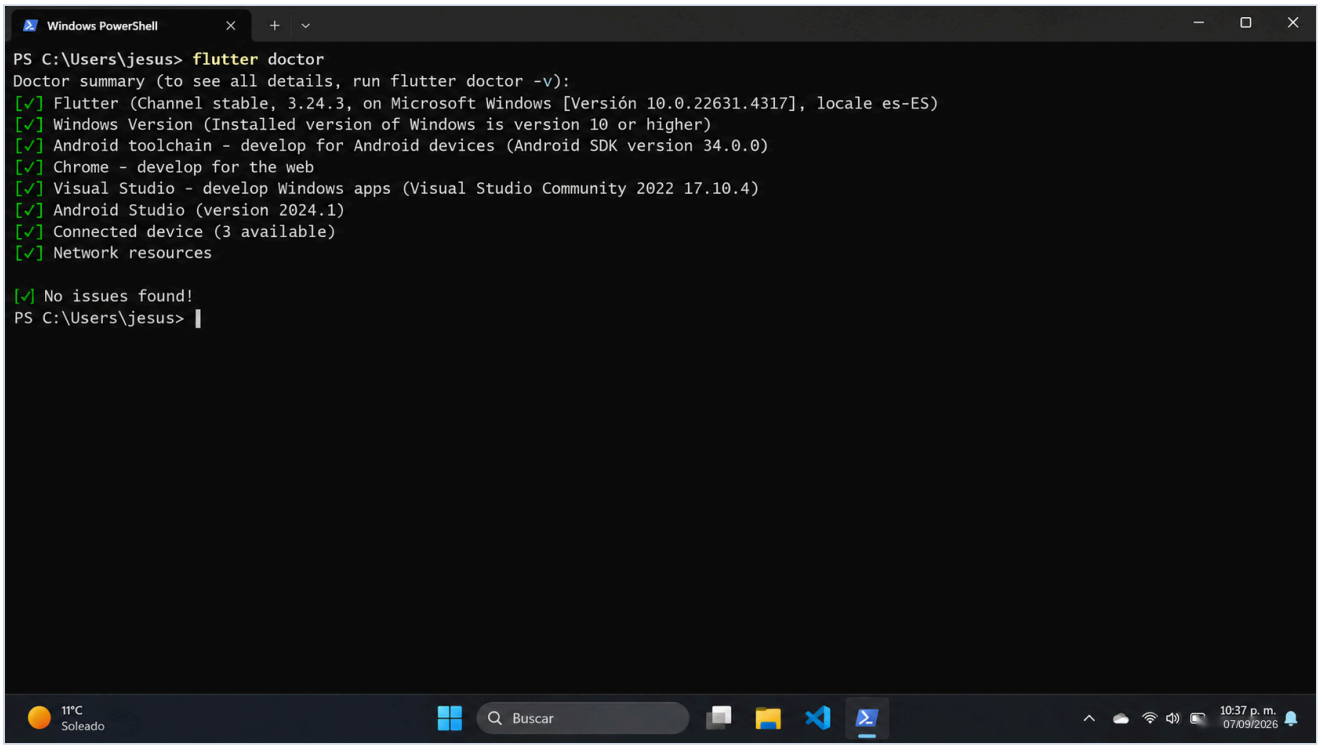
```
flutter clean
```

Elimina artefactos generados. Úsalo ante problemas de compilación que lo justifiquen, seguido de `flutter pub get`; no arregla errores de lógica.

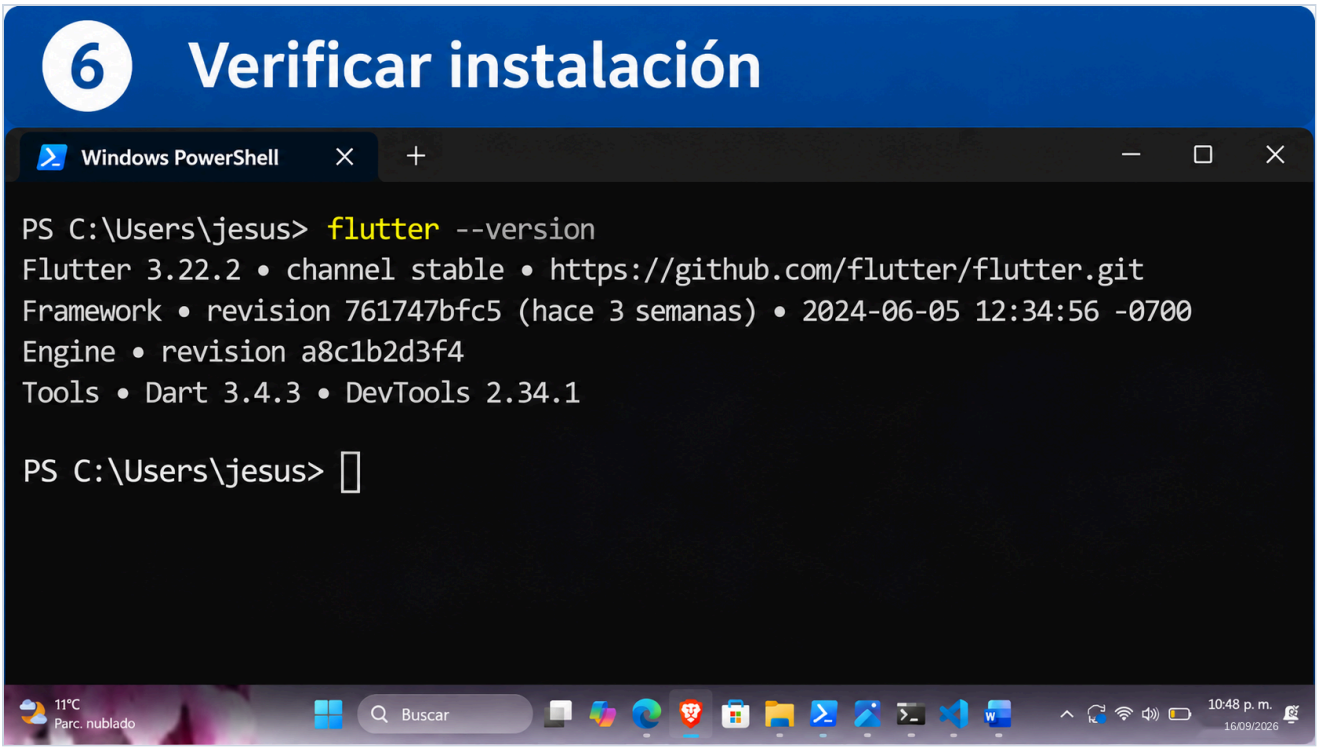
Capturas de verificación de Flutter



Consulta de las versiones de Flutter y Dart desde PowerShell.



Resultado de flutter doctor con las comprobaciones del entorno.



Ejemplo visual de flutter --version y del canal del SDK instalado.

Las versiones mostradas corresponden al entorno de las capturas; compara la salida de tu equipo con los requisitos del proyecto.

Ejemplo, verificación, errores y soluciones

Ejecuta el ejemplo del capítulo 16. Hot Reload (`r`) aplica muchos cambios sin reiniciar todo el estado; Hot Restart (`R`) lo reinicia. Si Flutter no se reconoce, revisa PATH y abre otra terminal.

E02: doctor y versiones. **Conclusión:** identifica qué comprobaciones corresponden a Android y cuáles a otras plataformas. Comandos Flutter.

Dart

¿Qué es, para qué sirve y papel arquitectónico?

Lenguaje de Flutter. Sus clases modelan datos y sus `Future` representan resultados que llegarán después, como una respuesta HTTP. Permite escribir UI y lógica de cliente; no guarda secretos del servidor.

Requisitos, instalación y configuración

Viene con Flutter. Utiliza el Dart de ese SDK para mantener compatibilidad; habilita su extensión en VS Code y revisa `environment.sdk` del proyecto.

Comandos principales

```
dart --version
```

```
dart format lib
```

Ejemplo práctico

El archivo `mobile/ejemplos/saludo.dart` define una función que muestra el nombre de una tarea. Desde `proyecto-movil`:

```
dart run mobile/ejemplos/saludo.dart
```

Verificación, errores comunes y soluciones

Debe imprimir `Tarea: Preparar entorno`. Si hay incompatibilidad de versión, compara la restricción del proyecto con el Dart incluido en Flutter antes de cambiarla.

Evidencia y conclusión

E17: versión, ejecución y explicación de `main`. Dart es el lenguaje; Flutter es el SDK de interfaz. Introducción oficial.

14. Android Studio y Android SDK

Android Studio: qué es, utilidad y arquitectura

IDE oficial para Android. En este laboratorio administra herramientas y dispositivos; VS Code puede seguir siendo tu editor. Requiere Windows compatible, espacio libre y acceso a Internet.

Instalación y configuración

Instala desde Android Studio, completa el asistente y abre **More Actions → SDK Manager** o **Tools → SDK Manager**. Anota **Android SDK Location**.

Android SDK: qué es y qué problema resuelve

Conjunto de herramientas que permite compilar, instalar y depurar en Android. Se utiliza entre Flutter y el dispositivo durante el desarrollo.

Componente	Función
SDK Platform	API de una versión Android usada al compilar
Build Tools	Construcción del paquete Android
Platform Tools	ADB y otras herramientas de comunicación
Command-line Tools	Administración por terminal, incluido sdkmanager
Android Emulator	Ejecución de dispositivos virtuales
NDK y CMake	Compilación de dependencias nativas cuando el proyecto las requiere

Instalación y configuración del SDK

En **SDK Platforms** instala la plataforma que solicita Flutter; la guía oficial consultada indica API 36. En **SDK Tools** instala Build-Tools, Platform-Tools, Command-line Tools, Emulator, NDK y CMake. Acepta cambios y espera las descargas. Configuración oficial de Flutter para Android.

Comandos principales y ejemplo práctico

```
flutter doctor --android-licenses
```

Lee y acepta las licencias necesarias. Después:

```
flutter doctor -v
```

Si usa otra carpeta del SDK:

```
flutter config --android-sdk "$env:LOCALAPPDATA\Android\Sdk"
```

Error especial: Android sdkmanager not found

Causas: falta Command-line Tools, instalación interrumpida o Flutter apunta a otro SDK. **Solución:** abre SDK Manager, instala **Android SDK Command-line Tools**, confirma la ubicación y verifica el ejecutable:

```
Test-Path "$env:LOCALAPPDATA\Android\Sdk\cmdline-tools\latest\bin\sdkmanager.bat"
```

Si tienes herramientas en un directorio versionado, usa esa ubicación real. Repite las licencias y el diagnóstico. No copies manualmente archivos entre SDK sin comprobar primero las rutas. Referencia sdkmanager.

Verificación, evidencia y conclusión

E18: SDK Manager, herramientas instaladas y licencias aceptadas. Android Studio y Android SDK son componentes distintos; instalar el IDE no demuestra por sí solo que las herramientas estén completas.

15. Emulador Android y dispositivo físico

Un **AVD** describe un dispositivo Android virtual: modelo, versión del sistema y recursos. El emulador lo ejecuta y permite probar sin un teléfono, a costa de más memoria y CPU.

Crear y verificar un AVD

1. Android Studio → **Device Manager** → **Create Virtual Device**.
2. Selecciona un teléfono y una imagen Android compatible con la arquitectura de tu CPU.
3. Descarga la imagen y asigna recursos razonables; para un equipo de 16 GB comienza con 2–4 GB de RAM para el emulador.
4. Finaliza y pulsa iniciar. Espera a que termine de arrancar Android.

```
flutter devices
```

Si no inicia, revisa virtualización y aceleración. La versión Android del teléfono emulado no tiene por qué ser idéntica al SDK usado para compilar, pero debe cumplir el mínimo de la aplicación. Administrar AVD.

Teléfono por USB

Activa opciones de desarrollador tocando repetidamente el número de compilación; habilita depuración USB, conecta un cable de datos y acepta la huella RSA de la computadora. Instala el controlador del fabricante si Windows lo necesita.

```
adb devices
```

`unauthorized` exige aceptar la autorización en el teléfono. `INSTALL_FAILED_USER_RESTRICTED` suele requerir permitir la instalación por USB o atender una restricción del dispositivo; no se resuelve modificando Dart. Dispositivos físicos.

E03: AVD o teléfono listado, pantalla Android y versión. Indica cuál usarás para la integración final.

16. Primer proyecto Flutter

En una carpeta de ejercicios fuera del repositorio actual:

```
flutter create app_demo
```

```
Set-Location app_demo
```

Ruta	Función
`lib/`	Código Dart de la aplicación
`lib/main.dart`	Entrada del programa
`android/`	Integración y compilación Android
`ios/`	Integración iOS; compilación en macOS
`web/`	Archivos de integración web
`test/`	Pruebas
`pubspec.yaml`	Dependencias, SDK y recursos

Reemplaza `lib/main.dart` por este ejemplo completo:

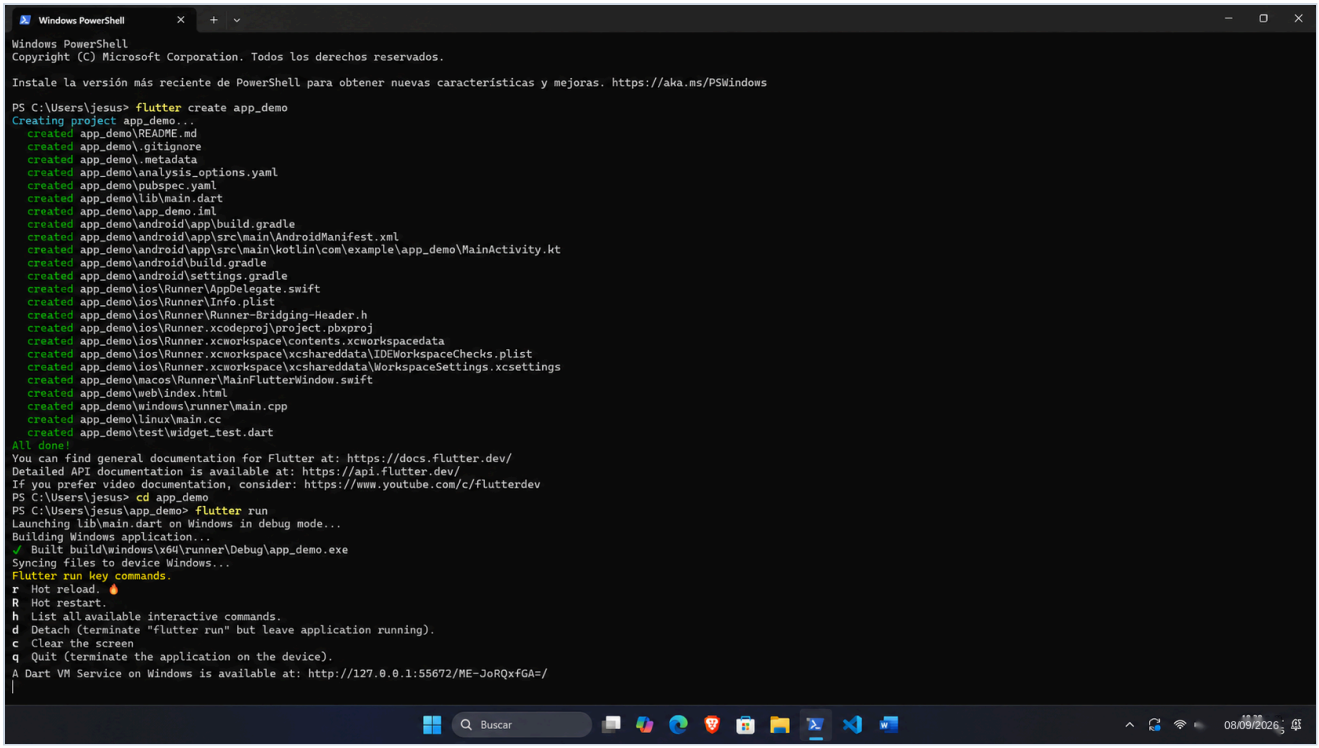
```
import 'package:flutter/material.dart';

void main() => runApp(const DemoApp());

class DemoApp extends StatelessWidget {
  const DemoApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: const Text('Mi entorno móvil')),
        body: const Center(child: Text('Flutter está funcionando')),
      ),
    );
  }
}
```

`runApp` inicia la UI; `MaterialApp` configura la aplicación; `Scaffold` organiza una pantalla. `const` identifica objetos inmutables construidos con valores constantes.

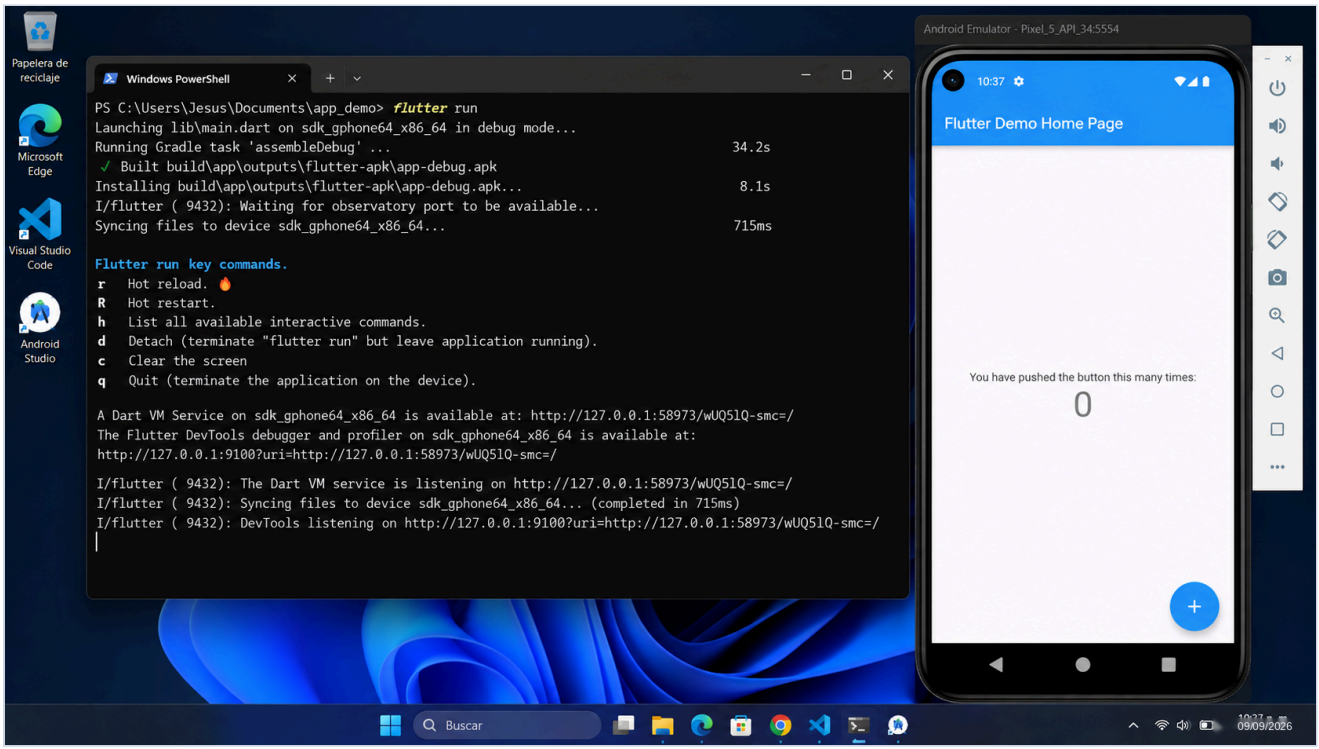
```
flutter run
```

Cambia el texto, guarda y pulsa `r`. Si ejecutas las pruebas generadas del contador, actualízalas para este nuevo diseño; su resultado esperado ya no es un contador.

Creación y ejecución en imágenes



Creación de un proyecto Flutter desde PowerShell y generación de sus archivos.



Ejecución del proyecto inicial de Flutter en un emulador Android.

La captura muestra el contador generado por Flutter antes de reemplazar lib/main.dart por el ejemplo de este capítulo.

E04: pantalla, código y recarga. El integrador completo está en `mobile/flutter_app`; este ejercicio introduce widgets antes de incorporar red y autenticación.

17. Fundamentos de HTTP y REST

HTTP define intercambios entre cliente y servidor. HTTPS agrega cifrado TLS para protegerlos en tránsito. Una **request** contiene método, dirección, encabezados y opcionalmente cuerpo; una **response** contiene estado, encabezados y cuerpo.

JSON representa objetos, listas, números, booleanos y texto. Ejemplo de tarea: `{"title":"Preparar entorno","completed":false}`. No admite comentarios ni comas finales.

Método	Intención	Ejemplo
GET	Consultar	`/tasks`
POST	Crear	`/tasks` con JSON
PUT	Reemplazar campos del recurso según contrato	`/tasks/3`
PATCH	Modificar parcialmente	No implementado en este proyecto
DELETE	Eliminar	`/tasks/3`

Estado	Significado en el laboratorio
200	Operación o consulta correcta
201	Recurso creado
400	Entrada inválida
401	Token ausente, inválido o vencido
403	Identidad válida sin el rol requerido
404	Recurso inexistente o fuera del alcance del usuario
409	Conflicto, por ejemplo correo repetido
500	Error inesperado del servidor
503	Dependencia temporalmente no disponible

REST es un estilo de diseño, no un formato de archivo. La API no confía en un `user_id` enviado por Flutter: obtiene el propietario del token validado. Los errores deben explicar el problema sin devolver contraseñas, trazas internas ni tokens.

Práctica: identifica método, URL, encabezados, cuerpo y estado en E07. Referencia HTTP.

18. Postman

¿Qué es, para qué sirve y papel arquitectónico?

Cliente HTTP para probar una API sin depender de la UI móvil. Ayuda a separar un fallo del backend de un problema de Flutter. No forma parte del servidor.

Requisitos, instalación y configuración

Instala la aplicación de escritorio desde Postman. Necesitas una API activa; retoma esta práctica después de levantar el backend.

Importa `postman/mobile.postman_collection.json` y `postman/local.postman_environment.json`. Una **Collection** agrupa solicitudes; un **Environment** guarda variables para un destino. Selecciona el entorno local; `{{base_url}}` representa `http://localhost:8000`.

Comandos principales y ejemplo práctico

Postman se opera con **Send**. Prueba primero `GET http://localhost:8000/`. Luego configura Authorization → Bearer Token con el token obtenido en el capítulo 25. Los endpoints `/usuarios` requieren administrador.

```
GET http://localhost:8000/usuarios
Authorization: Bearer TOKEN
```

```
POST http://localhost:8000/usuarios
Content-Type: application/json
Authorization: Bearer TOKEN
```

En **Body** → **raw** → **JSON**:

```
{"nombre":"Alex","correo":"alex@example.com"}
```

Headers describen la solicitud; Body contiene los datos. Un registro de `/usuarios` es un ejercicio de base de datos, no crea una cuenta Keycloak.

Verificación, errores comunes y soluciones

GET debe responder 200 y POST 201; repetir correo da 409. Si hay 401 revisa el token; si hay 403 revisa el rol. Postman de escritorio no aplica CORS como un navegador. No confundas un error de conexión con una respuesta HTTP.

Evidencia y conclusión

E07: método, URL, estado y JSON de respuesta, ocultando Authorization. Explica por qué probar primero la API reduce variables al depurar Flutter. Primeros pasos oficiales.

19. Alternativa A: Python y FastAPI

Python

¿Qué es, para qué sirve y papel arquitectónico?

Lenguaje interpretado con bibliotecas para web y datos. Ejecuta la lógica del backend de esta alternativa. Resuelve el desarrollo de servicios con código legible y tipado opcional.

Requisitos previos, instalación y configuración

Instala una versión compatible de Python para Windows. El contenedor usa la familia 3.13. Verifica el ejecutable y crea un entorno virtual: una carpeta que aísla las dependencias del proyecto.

```
python --version
```

Desde `backend`:

```
python -m venv .venv
```

Comandos y ejemplo práctico

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

Usar explícitamente el Python del entorno evita cambiar la política de ejecución de PowerShell. En una terminal con el entorno activado, la instalación mínima de una API educativa sería:

```
pip install fastapi uvicorn
```

Verificación, errores y soluciones

```
.\.venv\Scripts\python.exe -m pip --version
```

Si falta un módulo, comprueba que instalaste y ejecutas con el mismo intérprete. **E19:** versión, entorno y dependencias. Conclusión: el entorno virtual aísla paquetes, no es una máquina virtual. Referencia venv.

FastAPI

¿Qué es, para qué sirve y papel arquitectónico?

Framework de APIs en Python que aprovecha modelos tipados para validar y documentar entradas. **Uvicorn** es el servidor ASGI que recibe HTTP y ejecuta la aplicación. FastAPI es una alternativa completa a Express.

Requisitos, instalación y configuración

Utiliza el entorno anterior. La API real está en `backend/app/main.py`, con rutas, modelos, servicios y acceso a datos separados. Sus dependencias están en `requirements.txt`; Docker instala el mismo archivo.

```
backend/
├── app/
│   ├── main.py
│   ├── routes/api.py
│   ├── models/schemas.py
│   ├── services/auth.py
│   └── database/connections.py
├── tests/
├── requirements.txt
└── Dockerfile
```

Ejemplo inicial independiente

`backend/intro/main.py` implementa GET `/`, GET `/usuarios` y POST `/usuarios` en memoria. Desde `backend/intro`, con el entorno creado un nivel arriba:

```
..\.venv\Scripts\uvicorn.exe main:app --reload
```

Con Uvicorn en el PATH, el equivalente es:

```
uvicorn main:app --reload
```

Aquí los datos desaparecen al reiniciar. Detén este ejemplo antes de arrancar Compose, porque ambos usan 8000. La API integradora usa PostgreSQL y protege `/usuarios` con rol administrador.

Ejecutar la API integradora localmente

Levanta primero los servicios de datos y Keycloak con Compose. Desde `backend`, establece PGHOST, PGPORT, PGDATABASE, PGUSER, PGPASSWORD, MONGO_URL, MONGO_USER, MONGO_PASSWORD, KEYCLOAK_ISSUER y KEYCLOAK_JWKS_URL con valores de tu `.env`; cambia los hosts Docker por `localhost`. Los valores predeterminados no incluyen contraseñas.

```
.\.venv\Scripts\uvicorn.exe app.main:app --reload
```

La ruta recomendada para evitar diferencias de configuración es `docker compose up -d --build` desde la raíz del proyecto.

Verificación, errores comunes y soluciones

Abre `/docs`, consulta `/health` y prueba entradas inválidas. La API devuelve 400 en validación y 409 en duplicados. Si el módulo `app` no aparece, ejecuta desde `backend`; si falla una conexión, revisa el host interno o externo.

Evidencia y conclusión

E05: respuestas de creación y consulta; **E19:** pruebas Python. Explica la diferencia entre memoria y persistencia. Tutorial FastAPI.

20. Swagger y OpenAPI

¿Qué son y para qué sirven?

OpenAPI es un formato de contrato que describe rutas, entradas, respuestas y autenticación. **Swagger UI** es una interfaz que interpreta ese contrato y permite probar operaciones. Resuelven la falta de una referencia compartida entre frontend, backend y pruebas.

Papel, requisitos e instalación

Son parte de la documentación del backend. FastAPI genera OpenAPI automáticamente y publica Swagger y ReDoc. Express utiliza el contrato compartido en `docs/openapi.json` y el paquete `swagger-ui-express`; no genera documentación automáticamente a partir de cualquier ruta.

Configuración y comandos principales

Con la API activa, abre:

- `http://localhost:8000/docs`: Swagger UI en ambas alternativas.
- `http://localhost:8000/openapi.json`: contrato JSON.
- `http://localhost:8000/redoc`: ReDoc, disponible en FastAPI.

```
Invoke-RestMethod http://localhost:8000/openapi.json
```

Ejemplo práctico

En Swagger selecciona GET `/`, pulsa **Try it out** y **Execute**. Para `/tasks`, usa **Authorize** y pega el access token, sin el prefijo `Bearer` cuando la interfaz usa el esquema HTTP bearer. Ejecuta una creación con título válido y otra con título vacío.

Verificación, errores comunes y soluciones

Compara el contrato con la respuesta real. Si aparece 401, renueva el token; la documentación no evita autenticación. Si una ruta Express cambia, actualiza el contrato compartido y las pruebas. Un ejemplo documentado no demuestra que el endpoint funcione.

Evidencia y conclusión

E06: endpoint, esquema, solicitud y respuesta. Explica por qué OpenAPI y Swagger no son sinónimos. OpenAPI, documentación FastAPI.

21. Alternativa B: Node.js y Express

Node.js y npm

¿Qué son, para qué sirven y papel arquitectónico?

Node.js ejecuta JavaScript fuera del navegador. npm administra paquetes y scripts; `package.json` declara el proyecto y `package-lock.json` conserva la resolución de dependencias. Aquí ejecutan el backend alternativo a Python.

Requisitos, instalación y configuración

Instala la versión LTS desde Node.js. El contenedor usa Node 24. Abre otra terminal:

```
node --version
```

```
npm.cmd --version
```

En Windows `npm.cmd` evita problemas con la política de ejecución del envoltorio PowerShell. En otras terminales usa `npm`.

Comandos y ejemplo inicial

En una carpeta nueva de ejercicio:

```
npm init -y
```

```
npm install express
```

En el backend incluido, no regeneres el manifiesto: entra en `backend-node` y utiliza su archivo de bloqueo:

```
npm.cmd ci
```

Verificación, errores y soluciones

Si falta un paquete, instala desde la carpeta que contiene `package.json`. **E20:** versiones e instalación. Node es el runtime; npm es el administrador.

Express

¿Qué es, para qué sirve y papel arquitectónico?

Framework de rutas HTTP para Node.js. Un **middleware** recibe la solicitud antes de la respuesta: puede leer JSON, verificar un token o registrar un error. Permite construir la misma API REST de la alternativa FastAPI.

Requisitos, instalación y configuración

El proyecto usa Express 5, `pg`, el driver MongoDB, `jose` para JWT y Swagger UI. Las variables son las mismas que en FastAPI. Estructura:

```
backend-node/
├── src/
│   ├── config/settings.js
│   ├── controllers/api.js
│   ├── routes/api.js
│   ├── services/auth.js
│   ├── models/validation.js
│   └── app.js
├── tests/
├── package.json
└── Dockerfile
```

Comandos y ejemplo práctico

Desde `backend-node`, el ejemplo en memoria:

```
node intro/app.js
```

Ofrece GET `/`, GET `/usuarios` y POST `/usuarios`. Para el integrador, cambia `BACKEND_CONTEXT=../backend-node` en `.env` y reconstruye Compose tras detener el backend anterior. No debe existir otro proceso usando 8000.

```
npm.cmd test
```

Verificación, errores comunes y soluciones

Repite E05 y E06 para esta alternativa. `EADDRINUSE` indica puerto ocupado. Un error de JSON se responde 400; un correo repetido, 409. La validación no debe confiar en la UI.

Evidencia y conclusión

E20: endpoints y pruebas de Express. Compara la implementación con FastAPI: mismas responsabilidades y contrato, distintos lenguajes. Express 5.

22. PostgreSQL

¿Qué es y para qué sirve?

Motor relacional: organiza datos en tablas, cuyas filas representan registros y columnas atributos. Una clave primaria identifica una fila; una clave foránea conecta una fila con otra tabla. SQL permite definir y consultar esos datos.

Papel y requisitos

Conserva usuarios de aplicación, tareas y categorías. Resuelve integridad y relaciones; Keycloak guarda sus credenciales en una base separada. Requiere Docker y variables del capítulo 30.

Instalación y configuración

La ruta principal usa el contenedor `postgres:17`, una familia mantenida, y un volumen persistente. El script de inicialización crea un usuario de aplicación sin privilegios de superusuario y aplica `database/postgres/02-schema.sql`. No es necesario instalar otro servidor en Windows.

Para instalación nativa opcional utiliza el instalador oficial, selecciona puerto y contraseña, y evita ejecutarlo al mismo tiempo que el contenedor en 5432.

Comandos principales y ejemplo práctico

Desde `proyecto-movil`, con el servicio activo:

```
docker compose exec postgres psql -U postgres -d app_mobile
```

En un servidor nuevo sin la base, el comando de creación sería:

```
CREATE DATABASE app_mobile;
```

Compose ya la crea. En la sesión de `app_mobile`, practica con la tabla educativa `usuarios` (incluida en el esquema):

```
CREATE TABLE IF NOT EXISTS usuarios (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    correo VARCHAR(150) UNIQUE NOT NULL
);
```

```
INSERT INTO usuarios (nombre, correo) VALUES ('Alex', 'alex@example.com');
```

```
SELECT * FROM usuarios;
```

```
UPDATE usuarios SET nombre = 'Alex Rivera' WHERE correo = 'alex@example.com';
```

```
DELETE FROM usuarios WHERE correo = 'alex@example.com';
```

El backend conecta mediante `psycopg` o `pg`, usando consultas parametrizadas. El esquema integrador une `tasks.user_id` con `users.id` y `tasks.category_id` con `categories.id`.

Verificación, errores comunes y soluciones

Comprueba persistencia al reiniciar sin borrar volúmenes. `connection refused` exige revisar servicio, host y puerto; error de contraseña exige revisar credenciales. Cambiar `.env` no cambia usuarios de un volumen inicializado: modifica la contraseña con SQL o recrea un laboratorio descartable con autorización.

Evidencia y conclusión

E08: SELECT y relación entre tarea y propietario. Explica por qué una restricción UNIQUE sigue siendo necesaria aunque Flutter valide. Tutorial SQL.

23. MongoDB

¿Qué es y para qué sirve?

Base NoSQL orientada a documentos. Una colección agrupa documentos con campos; BSON es la representación binaria que admite tipos adicionales a JSON. Resuelve el almacenamiento de eventos con atributos que pueden variar, sin eliminar la necesidad de validar.

Papel, requisitos e instalación

Guarda `logs` y `activity`; PostgreSQL conserva las tareas. Se instala mediante `mongo:8.0` y un volumen. `database/mongodb/init.js` crea un usuario limitado a `app_mobile`. Instala MongoDB Compass si deseas una interfaz gráfica.

Configuración y comandos principales

Desde Windows, Compass conecta a `localhost:27017`, con el usuario de aplicación, su contraseña y base de autenticación `app_mobile`. Dentro de Docker, el host es `mongo`.

Si ya existe MongoDB local en 27017, añade `MONGO_PORT=27018` a `.env` y usa `localhost:27018` en Compass. El backend continúa usando `mongo:27017`; cambiar el puerto publicado no cambia el puerto interno.

```
docker compose exec mongo mongosh --username developer --authenticationDatabase app_mobile
app_mobile
```

Introduce la contraseña cuando la solicite. Si cambiaste el nombre de usuario, sustituye `developer`.

Ejemplo práctico

```
db.usuarios.insertOne({nombre: 'Alex', correo: 'alex@example.com', rol: 'usuario'})
```

```
db.usuarios.find({correo: 'alex@example.com'})
```

```
db.usuarios.updateOne({correo: 'alex@example.com'}, {$set: {nombre: 'Alex Rivera'}})
```

```
db.usuarios.deleteOne({correo: 'alex@example.com'})
```

Son operaciones CRUD: crear, leer, actualizar y eliminar. La colección `usuarios` es un ejercicio independiente de los eventos del integrador.

Verificación, errores comunes y soluciones

Después de crear una tarea consulta `db.activity.find()`. Si falla autenticación, revisa `authSource`, usuario y contraseña. Si aparece `connection refused`, comprueba el contenedor y el nombre de host. Las credenciales iniciales se aplican únicamente sobre un volumen vacío.

Evidencia y conclusión

E09: CRUD educativo y un evento real. Explica por qué la flexibilidad documental no significa ausencia de reglas. Instalación MongoDB, CRUD.

24. PostgreSQL frente a MongoDB

Aspecto	PostgreSQL	MongoDB
Modelo	Relacional, tablas	Documentos y colecciones
Relaciones	Claves foráneas y joins	Referencias o documentos embebidos
Estructura	Esquema explícito	Flexible, con validación opcional
Escalabilidad	Réplicas, partición y soluciones distribuidas	Réplicas y sharding
Casos de uso	Datos con integridad y relaciones	Documentos y eventos de estructura variable
Flexibilidad	Tipos, tablas y también JSON	Campos variables por documento
Consultas	SQL	Filtros y pipelines de agregación

Ninguna opción es automáticamente más rápida. El diseño de índices, volumen, consultas y consistencia determina el resultado. Ambas admiten funciones más amplias que esta comparación inicial.

En el integrador `users`, `tasks` y `categories` requieren relaciones claras. Los eventos en MongoDB contienen actor, acción, tarea y fecha. Usar dos bases añade operaciones y mantenimiento: se justifica aquí para practicar necesidades distintas, no como requisito de toda aplicación.

Actividad: justifica en cinco líneas dónde guardarías pagos y dónde eventos de navegación. No uses “NoSQL es mejor” como argumento; describe consultas y garantías necesarias.

25. Keycloak

¿Qué es, para qué sirve y papel arquitectónico?

Servicio de **IAM**, gestión de identidades y accesos. Centraliza cuentas, sesiones y roles para no implementar contraseñas en cada API. Autenticar identifica al usuario; autorizar limita sus acciones.

OAuth 2.0 delega acceso a recursos. **OpenID Connect** añade identidad sobre OAuth. Un **JWT** es un token firmado con claims; no está necesariamente cifrado. La API debe validarlo, no solo decodificarlo.

Concepto	Uso en este proyecto
Realm	Espacio de identidad `app-mobile`
Client	Aplicación registrada: `flutter-mobile`, `postman`
User	Cuenta de una persona
Role	`usuario` o `administrador`
Group	Agrupación opcional de personas y roles
Token	Credencial temporal emitida tras autenticación

Requisitos, instalación y configuración

Docker, base de identidad y `.env` configurado. Compose usa Keycloak 26.8.0 e importa `infrastructure/keycloak/app-mobile-realm.json`. Abre `http://localhost:8080`, entra con `KEYCLOAK_ADMIN` y `KEYCLOAK_ADMIN_PASSWORD` de tu `.env`.

1. Selecciona realm **app-mobile**.
2. En Realm roles comprueba `usuario` y `administrador`.
3. En Clients comprueba `flutter-mobile`: cliente público, Standard flow activo, Direct access grants desactivado, PKCE S256.
4. Comprueba redirect URI exacta `com.example.appmobile:/oauthredirect` y post logout URI `com.example.appmobile:/logout`.
5. En los mappers del cliente verifica audience `mobile-api` en access token.
6. En Users crea `alumno`, habilítalo y define contraseña en Credentials; desactiva **Temporary** para la práctica.
7. En Role mapping asigna `usuario`. Crea otra cuenta y asigna `administrador` para pruebas de administración.
8. Crea una tercera cuenta sin ninguno de esos roles para comprobar 403.

No confundas el administrador de la consola con un usuario de la aplicación. El realm se importa solo si no existe; editar el JSON no actualiza automáticamente un realm persistido.

Comandos y ejemplo práctico

```
docker compose logs keycloak
```

En Postman usa OAuth 2.0 → **Authorization Code (With PKCE)**, cliente `postman`, callback `https://oauth.pstmn.io/v1/callback`, sin client secret, scope `openid profile email` y challenge S256. URLs:

- Authorize: `http://localhost:8080/realms/app-mobile/protocol/openid-connect/auth`
- Token: `http://localhost:8080/realms/app-mobile/protocol/openid-connect/token`

Pulsa **Get New Access Token**, inicia sesión con el usuario y úsalo como Bearer. No uses password grant en Flutter ni copies contraseñas al código.

Verificación, errores comunes y soluciones

Revisa que `iss` coincida con `KEYCLOAK_ISSUER` y `aud` incluya `mobile-api`. Consulta `/me` y luego `/tasks`. Si no vuelve a Flutter revisa el redirect exacto. Si hay 401, revisa firma, emisor, audiencia y expiración; si hay 403, roles. El host público del emisor debe ser el mismo para el móvil y la API.

Evidencia y conclusión

E10: realm, clientes y roles; **E21:** login y acceso permitido/rechazado sin mostrar tokens. Contenedor, OIDC, hostname.

26. Flujo de autenticación

```
sequenceDiagram
  actor U as Usuario
  participant F as Flutter
  participant K as Keycloak
  participant A as API REST
  participant P as PostgreSQL
  participant M as MongoDB
  U->>F: Iniciar sesión
  F->>K: Navegador: authorization code + PKCE
  K->>U: Solicitar credenciales
  U->>K: Autenticarse
  K-->>F: Código mediante redirect
  F->>K: Canjear código con verifier PKCE
  K-->>F: Access token e ID token
  F->>A: Bearer access token y solicitud
  A->>A: Validar firma, iss, aud, exp y rol
  A->>P: Operación del propietario
  A->>M: Registrar actividad
  A-->>F: JSON
  F-->>U: Actualizar interfaz
```

```
Authorization: Bearer TOKEN
```

El access token autoriza acceso a la API. El ID token informa del inicio de sesión al cliente: no se usa en lugar del access token. PKCE vincula el código al cliente que inició el flujo, sin requerir un secreto incrustado en Flutter.

Cada petición se valida de forma independiente. El cliente oculta botones por comodidad, pero el backend sigue comprobando roles y propiedad. Una cuenta sin rol obtiene 403; una solicitud sin token obtiene 401; una tarea de otro propietario devuelve 404 para no revelar su existencia.

Cerrar sesión borra las credenciales locales e intenta finalizar la sesión del navegador en Keycloak. Un access token ya emitido puede ser válido hasta su expiración; la API no consulta revocación por cada solicitud en este laboratorio.

27. Docker

¿Qué es y para qué sirve?

Ejecuta procesos aislados a partir de imágenes. Reduce diferencias entre computadoras al empaquetar dependencias. Una máquina virtual incluye su propio sistema/kernel; los contenedores Linux comparten el kernel del entorno Linux que Docker Desktop administra en Windows.

Concepto	Explicación
Imagen	Plantilla de archivos y configuración
Contenedor	Instancia en ejecución o detenida
Dockerfile	Instrucciones para construir una imagen
Volumen	Datos persistentes fuera del ciclo de vida del contenedor
Puerto	Punto de escucha TCP/UDP
Network	Red que comunica contenedores
Registry	Servicio de distribución de imágenes

Papel, requisitos, instalación y configuración

Ejecuta el backend y sus dependencias. Requiere virtualización, WSL 2 y Windows compatible. Instala Docker Desktop, completa WSL si se solicita, reinicia e inicia Docker Desktop con contenedores Linux. Espera a que el motor esté disponible.

Comandos principales y ejemplo práctico

```
docker --version
```

```
docker version
```

El segundo debe mostrar Client y Server.

```
docker pull hello-world
```

```
docker run --name prueba-docker hello-world
```

```
docker ps
```

```
docker ps -a
```

```
docker images
```

```
docker logs prueba-docker
```

`hello-world` termina inmediatamente. Para detener un contenedor de larga duración usa `docker stop NOMBRE`. Elimina solo el ejemplo detenido:

```
docker rm prueba-docker
```

Verificación, errores comunes y soluciones

Si solo aparece Client, inicia el motor. Si falla WSL, consulta su estado y virtualización. No borres todos los volúmenes para solucionar un puerto ocupado; identifica el proceso.

Evidencia y conclusión

E11: Client/Server y hello-world. Explica por qué una imagen no equivale a un contenedor. Conceptos Docker.

28. Dockerfile

Un Dockerfile expresa la construcción de una imagen en capas. Cambiar código puede reutilizar la capa de dependencias si se copiaron primero sus manifiestos.

Instrucción	Función
FROM	Imagen base
WORKDIR	Directorio de trabajo
COPY	Copiar archivos desde el contexto
RUN	Ejecutar una acción durante construcción
EXPOSE	Documentar puerto; no lo publica por sí solo
CMD	Comando predeterminado al iniciar

FastAPI: `backend/Dockerfile`

Utiliza Python 3.13 slim, instala `requirements.txt`, copia `app`, crea un usuario sin privilegios y ejecuta Uvicorn en `0.0.0.0:8000`. Revisa el archivo real: es el usado por Compose.

Express: `backend-node/Dockerfile`

Utiliza Node 24 slim, instala con `npm ci --omit=dev`, copia `src` y el contrato OpenAPI, selecciona el usuario `node` y ejecuta `node src/server.js`.

Construye desde la raíz:

```
docker build -t mobile-fastapi ./backend
```

```
docker build -t mobile-express ./backend-node
```

Los archivos `.dockerignore` excluyen entornos locales, caches y secretos. Las variables de ejecución se entregan al contenedor; no copies `.env` a una imagen. El servidor escucha en todas las interfaces del contenedor, mientras Compose limita los puertos publicados al equipo local.

E22: construcción de la alternativa elegida y explicación de sus capas. Un build correcto comprueba la imagen, no la conectividad completa con las bases. Referencia Dockerfile.

29. Docker Compose

¿Qué es, para qué sirve y papel arquitectónico?

Herramienta para describir varios contenedores como un sistema. Resuelve el arranque manual con numerosos comandos y redes inconsistentes.

Requisitos e instalación

Docker Desktop incluye Compose v2. Comprueba:

```
docker compose version
```

Configuración

`infrastructure/docker-compose.yml` contiene backend, PostgreSQL, MongoDB, Keycloak y un PostgreSQL independiente para identidad. `services` define procesos; `volumes` preserva datos; `networks` permite comunicación por nombres; `environment` configura procesos; `ports` publica servicios al anfitrión.

Desde `proyecto-movil`, copia `.env.example` a `.env`, sustituye contraseñas y conserva `COMPOSE_FILE=infrastructure/docker-compose.yml`. Las rutas de montaje y build se resuelven respecto del archivo Compose; por eso usan `../backend` y `../database`.

Comandos principales y ejemplo práctico

```
docker compose config --quiet
```

Valida la configuración sin imprimir valores sensibles.

```
docker compose up -d
```

```
docker compose ps
```

```
docker compose logs --tail 50 backend
```

```
docker compose build
```

```
docker compose down
```

Usa `up -d --build` después de cambiar código. Para Express modifica `BACKEND_CONTEXT` según README y reconstruye. Los healthchecks de las bases evitan iniciar el backend antes de que respondan; Keycloak puede tardar más. `/health` comprueba las dos bases, no sustituye la prueba del login.

Verificación, errores comunes y soluciones

Si 8000 está ocupado, detén tu Uvicorn/Node local. Si hay un volumen antiguo, los scripts de inicialización no vuelven a correr. `down` conserva volúmenes; `down -v` borra datos y requiere asumir esa pérdida.

Aplicación paso a paso en este proyecto

El archivo real es docker-compose.yml. Compose organiza los siguientes cinco servicios de la aplicación de tareas:

Servicio	Origen	Función y acceso desde Windows
`backend`	Construye `../backend/Dockerfile` por defecto	API FastAPI en `http://localhost:8000`; Swagger en `/docs`
`postgres`	`postgres:17`	Datos de la aplicación, puerto `5432`
`mongo`	`mongo:8.0`	Eventos de actividad, puerto `27017` por defecto
`keycloak-db`	`postgres:17`	Base de identidad; sin puerto publicado a Windows
`keycloak`	`quay.io/keycloak/keycloak:26.8.0`	Autenticación en `http://localhost:8080`

Flutter se ejecuta en Android y consume la API; no hay un contenedor Flutter en este archivo. Los servicios comparten la red `mobile`: el backend encuentra las bases por los nombres `postgres` y `mongo`, y consulta las claves de autenticación en `keycloak:8080`. Dentro de un contenedor, `localhost` designa ese mismo contenedor.

Figura 29.1. Guía ilustrada de los cinco servicios definidos en el proyecto; no representa una captura de ejecución.

1. Preparar Docker y la carpeta de trabajo

Abre Docker Desktop y espera a que el motor de contenedores Linux esté listo. Desde la raíz del repositorio ejecuta en PowerShell:

```
Set-Location .\Manual-Flutter\proyecto-movil
docker version
docker compose version
```

`docker version` debe mostrar Client y Server. Todos los comandos siguientes se ejecutan desde `proyecto-movil`.

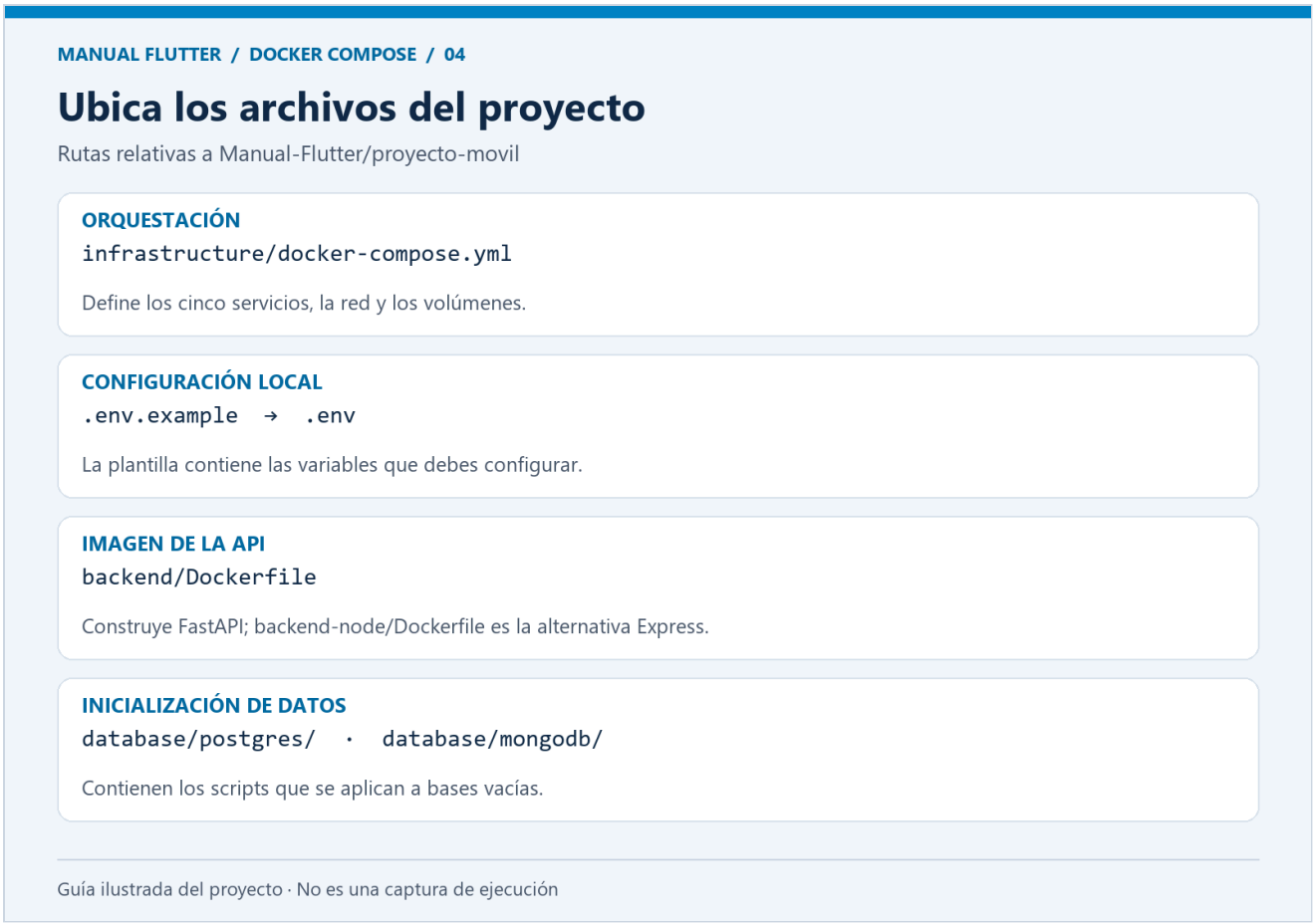


Figura 29.2. Archivos del proyecto que intervienen en la configuración, construcción e inicialización.

2. Crear la configuración local

Este comando copia la plantilla únicamente si todavía no existe `.env`, para conservar una configuración previa:

```
if (-not (Test-Path .env)) { Copy-Item .env.example .env }
notepad .env
```

La plantilla `.env.example` ya contiene estas opciones:

```
COMPOSE_FILE=infrastructure/docker-compose.yml
COMPOSE_PROJECT_NAME=manual_mobile
BACKEND_CONTEXT=../backend
POSTGRES_DB=app_mobile
POSTGRES_USER=developer
MONGO_PORT=27017
```

Conserva las demás variables de la plantilla y sustituye los valores de `POSTGRES_PASSWORD`, `POSTGRES_ADMIN_PASSWORD`, `MONGO_PASSWORD`, `MONGO_ROOT_PASSWORD`, `KEYCLOAK_DB_PASSWORD` y `KEYCLOAK_ADMIN_PASSWORD` antes del primer arranque. Guarda `.env`; no lo incluyas en Git ni en capturas.

`COMPOSE_FILE` selecciona el archivo de infraestructura y `COMPOSE_PROJECT_NAME` agrupa los recursos como `manual_mobile`. Las rutas `../backend` y `../database` parten de la carpeta `infrastructure`. Compose lee las variables y el YAML pasa explícitamente al contenedor las que necesita mediante `environment`.

Figura 29.3. Variables de la plantilla y pasos para preparar `.env`, sin mostrar contraseñas.

3. Validar y arrancar

```
docker compose config --quiet
docker compose config --services
docker compose up -d --build
docker compose ps -a
```

Continúa al arranque solo si la validación termina sin errores. `config --services` debe listar los cinco nombres de la tabla. `up -d --build` construye la imagen del backend, obtiene las imágenes necesarias y crea contenedores, red y volúmenes. La primera ejecución necesita acceso a Internet y puede tardar varios minutos.

Las bases de datos tienen `healthcheck`. El backend espera a que PostgreSQL y MongoDB estén saludables; Keycloak espera a su propia base. La condición `service_started` de Keycloak no garantiza que el login ya esté listo: revisa su registro antes de probar autenticación.

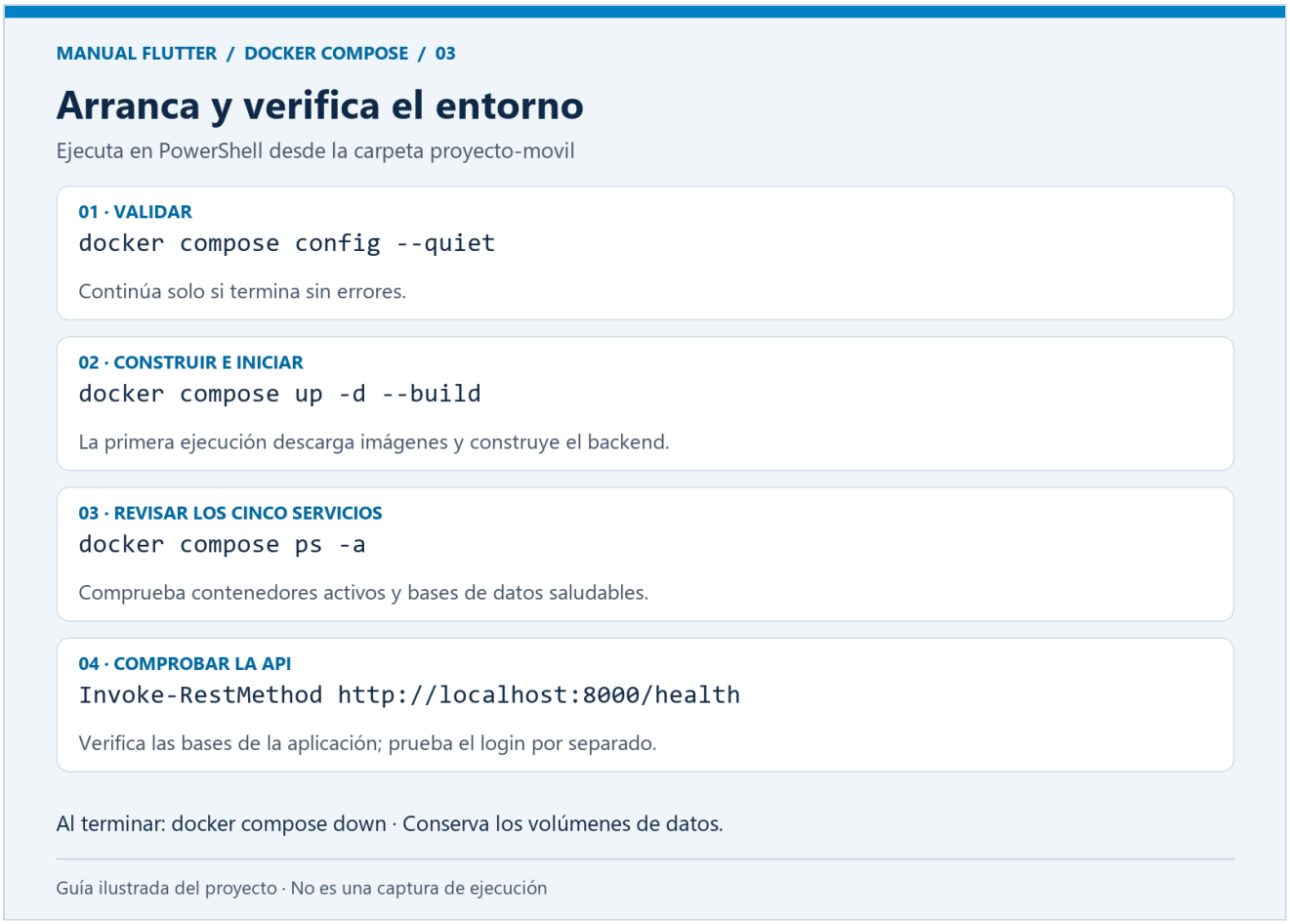


Figura 29.4. Guía ilustrada de arranque y verificación; las salidas reales deben registrarse en la evidencia E12.

4. Comprobar el resultado

```
docker compose logs --tail 50 backend keycloak
Invoke-RestMethod -Uri http://localhost:8000/health
```

Comprueba que los cinco contenedores siguen activos y que las bases muestran `healthy`. Abre `http://localhost:8000/docs` para revisar la API FastAPI y `http://localhost:8080` para acceder a Keycloak. `/health` verifica las dos bases de la aplicación; el login se comprueba por separado siguiendo el capítulo 25.

Para conectar Flutter en un teléfono Android por USB, configura ADB y las URLs según el README del proyecto y el capítulo 32. Los puertos de este Compose se publican en `127.0.0.1`: el teléfono no puede acceder directamente mediante la IP de la computadora en la red Wi-Fi.

Figura 29.5. Puntos de comprobación desde Windows. La guía no representa resultados de una ejecución real.

5. Conservar datos y administrar los servicios

Acción	Comando desde `proyecto-movil`
Consultar estado	`docker compose ps -a`
Seguir el registro del backend	`docker compose logs -f backend` (Ctrl+C termina la vista del registro)
Reconstruir después de editar el backend	`docker compose up -d --build backend`
Detener temporalmente	`docker compose stop`
Volver a iniciar contenedores detenidos	`docker compose start`
Eliminar contenedores y red al terminar	`docker compose down`
Volver a crear el entorno	`docker compose up -d --build`

Los volúmenes `postgres_data`, `mongo_data` y `keycloak_data` conservan los datos al ejecutar `down`. Los archivos de `database` se montan para inicializar bases vacías; modificarlos no actualiza automáticamente una base ya creada. Cambiar las contraseñas en `.env` tampoco cambia por sí solo las contraseñas guardadas en esas bases. `docker compose down -v` elimina los volúmenes y sus datos: no forma parte del cierre habitual de esta práctica.

Para utilizar la alternativa Express, sigue el cambio de `BACKEND_CONTEXT=../backend-node` y reconstrucción descrito en el README. El resto de servicios se mantiene y se utiliza un solo backend a la vez.

Figura 29.6. Diferencias entre detener servicios, cerrar el entorno y eliminar datos persistentes.

Problemas específicos de este entorno

Síntoma	Revisión y solución
No encuentra el archivo de configuración	Ejecuta desde `proyecto-movil` y comprueba que `.env` conserva `COMPOSE_FILE`
Solicita una variable obligatoria	Completa la variable indicada en `.env` y repite `config --quiet`
No conecta con el motor	Inicia Docker Desktop y confirma que `docker version` muestra Server
Puerto 8000, 5432 o 8080 ocupado	Identifica el servicio local que usa ese puerto y detenlo si corresponde antes de arrancar Compose
Puerto 27017 ocupado	Ajusta `MONGO_PORT` en `.env`; la conexión interna del backend sigue usando `mongo:27017`
Un servicio termina o queda no saludable	Consulta `docker compose ps -a` y `docker compose logs --tail 100 NOMBRE_SERVICIO`
La API responde pero el login falla	Revisa Keycloak, el realm `app-mobile`, los usuarios y las URLs del capítulo 32

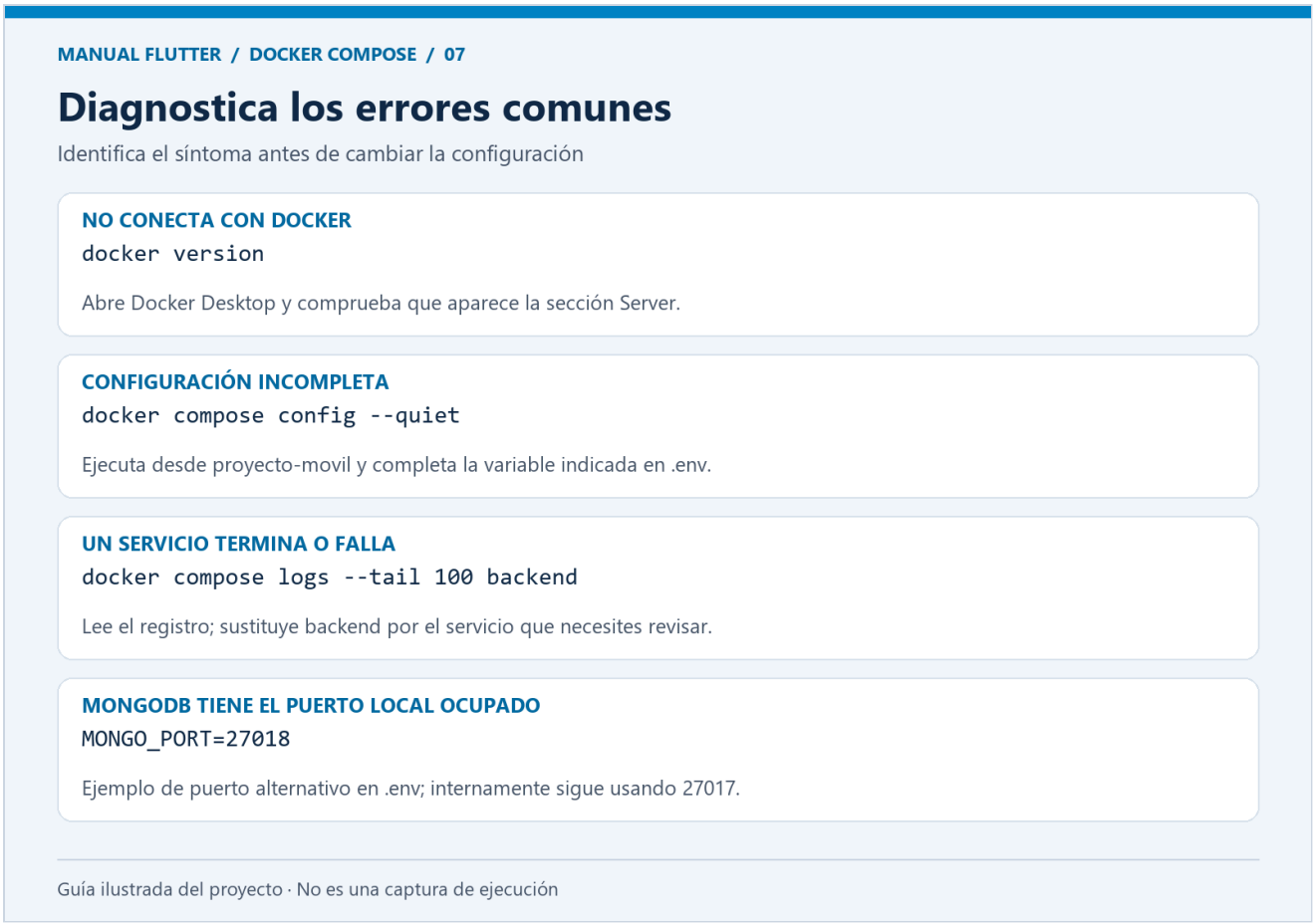


Figura 29.7. Guía de diagnóstico. El puerto 27018 es un ejemplo alternativo: comprueba que esté disponible y vuelve a ejecutar `docker compose up -d` para aplicar el cambio.

Evidencia y conclusión

Para la evidencia E12, registra la versión de Compose, los cinco servicios, su estado y la respuesta real de `/health`. Añade capturas de Swagger y Keycloak cuando ambos respondan. No presentes estos resultados esperados como pruebas ejecutadas.

Referencias: variables de Compose, comandos de Compose y volúmenes.

E12: servicios, health y arranque. Explica diferencia entre ordenar arranque y garantizar disponibilidad permanente. Compose, dependencias y salud.

30. Variables de entorno y secretos

`.env` contiene configuración local de Compose. `.env.example` documenta los nombres y valores iniciales sin secretos reales. `.gitignore` excluye el archivo real; no elimina secretos que ya se hayan publicado.

Desde `proyecto-movil`:

```
Copy-Item .env.example .env
```

Antes de iniciar sustituye cada valor `cambiar_...` por una contraseña propia. Para este laboratorio puedes generar cadenas hexadecimales con tu gestor de contraseñas; así evitas problemas de caracteres reservados en URLs y scripts.

Variable	Significado
POSTGRES_DB	Base de aplicación: `app_mobile`
POSTGRES_USER	Usuario limitado del backend: `developer`
POSTGRES_PASSWORD	Contraseña de ese usuario
POSTGRES_ADMIN_PASSWORD	Administración del contenedor PostgreSQL
MONGO_URL	Dirección interna: `mongodb://mongo:27017/app_mobile`
MONGO_USER / MONGO_PASSWORD	Autenticación de la aplicación MongoDB
KEYCLOAK_URL	Dirección interna de identidad: `http://keycloak:8080`
KEYCLOAK_PUBLIC_URL	Dirección del navegador: `http://localhost:8080`
BACKEND_CONTEXT	`../backend` o `../backend-node`

Los drivers reciben usuario/contraseña Mongo por separado; `MONGO_URL` sin contraseña no desactiva la autenticación. El backend recibe las variables expresamente enumeradas en Compose. Una variable en `.env` no entra automáticamente a todos los contenedores.

```
git check-ignore .env
```

Debe mostrar `.env`. No compartas la salida completa de `docker compose config`, pues puede incluir secretos. Si publicaste uno, rótalo aunque borres el archivo.

E23: nombres de variables y prueba de exclusión, con valores ocultos. Variables en Compose.

31. Integración Flutter y API

```
flowchart TD
  UI[Flutter UI: main.dart] --> Controller[TasksController: estado y loading]
  Controller --> Service[ApiService: GET / POST / PUT / DELETE]
  Service --> Client[Cliente http: JSON, timeout y Bearer]
  Client --> API[API REST: FastAPI o Express]
```

Cada capa tiene una responsabilidad. La UI presenta y recibe acciones; el controlador actualiza estado; el servicio realiza solicitudes. Se usa `ChangeNotifier` sin añadir otro administrador de estado para mantener visible el flujo.

Archivos reales

Archivo en `mobile/flutter_app`	Responsabilidad
`lib/main.dart`	Login, lista, formulario, confirmación y perfil
`lib/controllers/tasks_controller.dart`	`busy`, errores, carga y mutaciones
`lib/services/api_service.dart`	Cliente HTTP, códigos, JSON y timeout
`lib/models/task.dart`	Conversión de JSON a Task
`lib/config.dart`	URLs no secretas recibidas al compilar

Desde esa carpeta:

```
flutter pub get
```

El paquete `http` está declarado en `pubspec.yaml`. El servicio admite llamadas como las utilizadas por el controlador:

```
final values = await api.request('GET', '/tasks') as List;
```

```
await api.request('POST', '/tasks', body: {
  'title': 'Preparar demostración',
  'completed': false,
  'category_id': null,
});
```

Estas expresiones pertenecen a métodos del controlador; el programa completo está en los archivos señalados. `jsonEncode` serializa el cuerpo; `jsonDecode` convierte la respuesta. `await` espera el resultado sin bloquear toda la interfaz.

Errores y carga

El indicador de carga aparece mientras `busy` es verdadero y deshabilita mutaciones simultáneas. La solicitud tiene timeout de 15 segundos. Un error de red, JSON inválido o estado HTTP fallido se convierte en un mensaje. Ante 401 el servicio intenta renovar el token una vez; no repite indefinidamente.

> **Nota:** si el servidor guardó una tarea y la conexión se cortó antes de recibir la respuesta, reintentar POST puede duplicarla. Consulta la lista antes de repetir; un sistema productivo puede agregar claves de idempotencia.

Verificación y evidencia

E14: crear desde Flutter, comprobar la respuesta y verla en PostgreSQL. **E24:** pruebas del servicio HTTP. Prueba también la API apagada y confirma que aparece un error en lugar de una carga infinita. Paquete http.

32. Integración Flutter y Keycloak

Secuencia implementada

1. Flutter abre el navegador mediante `flutter_appauth`.
2. Keycloak autentica; AppAuth canjea el código con PKCE.
3. `AuthService` guarda tokens en `flutter_secure_storage`, no en texto dentro del repositorio.
4. `ApiService` agrega el access token al encabezado Bearer.
5. La API valida RS256, firma JWKS, `iss`, `aud`, `exp` y presencia de `sub` e `iat`.
6. Los roles y el propietario determinan qué recurso puede utilizarse.

`401` indica autenticación no aceptada. `403` indica identidad aceptada sin permisos suficientes. La UI no puede concederse roles modificando JSON.

Ruta recomendada: Android por USB o emulador con ADB

Con Compose iniciado, agrega Platform Tools al PATH y ejecuta:

```
adb devices
```

```
adb reverse tcp:8000 tcp:8000
```

```
adb reverse tcp:8080 tcp:8080
```

```
adb reverse --list
```

Si hay varios dispositivos, usa `adb -s ID_DEL_DISPOSITIVO reverse ...`. Este puente hace que `localhost:8000` y `localhost:8080` del Android lleguen a la computadora. Debes restablecerlo tras desconectar o reiniciar.

Conserva `KEYCLOAK_PUBLIC_URL=http://localhost:8080` en `.env`. Desde `mobile/flutter_app`:

```
flutter run -d ID_DEL_DISPOSITIVO --dart-define=API_URL=http://localhost:8000 --dart-define=ISSUER=http://localhost:8080/realms/app-mobile
```

El emulador Android también puede acceder al anfitrión con `10.0.2.2`, pero no mezcles un issuer `localhost` con otro `10.0.2.2`: el emisor del token debe coincidir exactamente. La ruta ADB evita esa diferencia para esta práctica.

Configuración Android incluida

- Permiso INTERNET en el manifest principal.

- `appAuthRedirectScheme=com.example.appmobile` en Gradle.
- Callback exacto registrado en Keycloak.
- HTTP permitido solo en el manifest **debug** y en AppAuth cuando `kDebugMode` es verdadero.
- Copias de seguridad deshabilitadas para evitar restaurar credenciales cifradas en otro contexto.

Alternativa por Wi-Fi

El teléfono necesita la IP LAN de la computadora, puertos accesibles por firewall y un issuer público consistente. Compose publica por defecto en `127.0.0.1`; para una LAN de laboratorio debes cambiar esos enlaces conscientemente. No basta con cambiar la URL Flutter. Para despliegues reales utiliza DNS, HTTPS, proxies y un Keycloak configurado para producción.

Verificación, errores y evidencia

Inicia sesión, abre perfil, crea una tarea y cierra sesión. Después de cinco minutos comprueba renovación o reautenticación. Cancelar el navegador debe volver a la pantalla sin bloquearla. **E21:** flujo sin mostrar credenciales; **E14:** respuesta en Flutter. El cierre local siempre borra la sesión; si no se puede cerrar el navegador, aparece un aviso.

Fuentes: Flutter AppAuth, almacenamiento seguro, ADB.

33. Plan de pruebas

Prueba	Herramienta	Resultado esperado
Entorno Flutter	`flutter doctor -v`	Sin pendientes que impidan Android
Dispositivo	`flutter devices`	Android disponible
API activa	GET `/`	200 y nombre de backend
Dependencias	GET `/health`	PostgreSQL y MongoDB ok
Swagger	Navegador `/docs`	Contrato y ejecución
PostgreSQL	psql SELECT	Tarea persistida con propietario
MongoDB	mongosh find	Eventos create/update/delete
Keycloak	Discovery OIDC	Issuer y claves disponibles
Token válido	GET `/tasks`	200 con rol adecuado
Token ausente/alterado/vencido	Postman	401
Rol insuficiente	Cuenta sin roles	403
Otro propietario	PUT tarea de otra cuenta	404 y ningún cambio
Datos inválidos	POST título vacío	400
Correo duplicado	POST `/usuarios` dos veces	409 en segundo intento
Docker	`docker version`	Client y Server
Compose	`docker compose ps`	Servicios activos
Flutter y API	Crear y recargar	Tarea visible
Persistencia	Reiniciar servicios	Tarea conservada

Pruebas automatizadas

Desde `backend`:

```
.\.venv\Scripts\python.exe -m pytest -q
```

Desde `backend-node`:

```
npm.cmd test
```

Desde `mobile/flutter_app`:

```
flutter analyze
```

```
flutter test
```

Estas pruebas comprueban validación, JWT y manejo de solicitudes con dependencias controladas. No reemplazan las pruebas con contenedores reales ni la interacción del navegador OAuth.

Desde `proyecto-movil` puedes ejecutar la prueba de integración después de obtener un token de usuario en Postman. El script solicita el token sin imprimirlo:

```
python scripts/smoke.py
```

El script crea una tarea propia, la edita y la elimina; comprueba 401 y entrada inválida. No uses datos de producción. Para comprobar permisos entre usuarios, sigue la tabla manual con dos cuentas diferentes.

Consulta VERIFICACION para distinguir resultados ejecutados de resultados todavía pendientes. **E24:** comando, resultado real y alcance de la prueba.

34. Estructura del repositorio

```
proyecto-movil/
├── mobile/
│   ├── ejemplos/saludo.dart
│   └── flutter_app/
│       ├── android/
│       ├── lib/
│       └── test/
├── backend/
│   ├── app/
│   ├── intro/
│   ├── tests/
│   └── Dockerfile
├── backend-node/
│   ├── src/
│   ├── intro/
│   ├── tests/
│   └── Dockerfile
├── database/
│   ├── postgres/
│   └── mongodb/
├── infrastructure/
│   ├── keycloak/
│   └── docker-compose.yml
├── docs/
│   ├── diagrams/
│   ├── installation/
│   └── evidence/
├── postman/
├── scripts/
├── .gitignore
├── .env.example
└── README.md
```

`backend` y `backend-node` no dependen entre sí en ejecución. `intro` contiene ejercicios en memoria; `app/src` contienen el integrador. Las bases se inicializan con scripts versionados y mantienen datos en volúmenes, nunca en archivos SQL con contraseñas reales.

Los tests viven junto al componente que validan. Los archivos de bloqueo de npm y Flutter deben viajar con el proyecto para repetir sus dependencias. Los diagramas Mermaid pueden abrirse desde los capítulos o reutilizarse en una presentación.

Este proyecto está dentro del repositorio de prácticas; no se crea un repositorio Git anidado. Si deseas distribuirlo de forma independiente, copia su carpeta fuera y sigue el flujo de GitHub del capítulo 12.

35. Flujo de ramas Git

```
main
└── develop
    ├── feature/flutter
    ├── feature/api
    ├── feature/database
    ├── feature/keycloak
    └── feature/docker
```

Rama	Propósito
main	Entregas estables verificadas
develop	Integración de trabajo del equipo
feature/flutter	UI y consumo de API
feature/api	Backend y contrato
feature/database	Esquemas y consultas
feature/keycloak	Identidad y roles
feature/docker	Imágenes y Compose

Ejemplo desde un repositorio de práctica con `main`:

```
git switch main
```

```
git switch -c develop
```

```
git switch -c feature/api
```

Registra cambios y publica la rama. Crea un PR hacia `develop`, exige pruebas y revisión; luego crea un PR de `develop` hacia `main` para la entrega.

```
git push -u origin feature/api
```

No recrees una rama que ya existe: usa `git switch NOMBRE`. En este repositorio el material se elabora en `Manual-Flutter`; el árbol anterior es una propuesta para organizar el proyecto académico, no una afirmación de ramas ya creadas.

36. Seguridad

- No publiques contraseñas, access tokens, refresh tokens ni archivos `.env` reales. Si se filtran, rótalos.
- Utiliza HTTPS en producción; el HTTP y `start-dev` de este laboratorio están limitados al desarrollo.
- Valida entradas en el backend aunque Flutter ya las revise.
- Utiliza consultas SQL parametrizadas; nunca formes SQL concatenando texto del usuario.
- Verifica firma, algoritmo permitido, emisor, audiencia y expiración de JWT.
- Asigna roles mínimos y comprueba el propietario en cada operación de tareas.
- Mantén corta la duración de access tokens; trata refresh tokens como credenciales sensibles.
- No coloques secretos de cliente en Flutter. Un binario distribuido puede inspeccionarse.
- Separa la base de identidad de la base de negocio y usa usuarios de aplicación con permisos limitados.
- Conserva puertos de desarrollo en loopback; no publiques bases de datos a Internet.
- Actualiza dependencias, revisa cambios y vuelve a ejecutar pruebas antes de promover versiones.
- Excluye tokens de capturas, logs, colecciones exportadas y ejemplos de soporte.

El laboratorio no incluye recuperación de cuenta, correo, TLS, proxy, protección contra fuerza bruta configurable para producción ni un sistema de auditoría transaccional. Antes de un despliegue real deben diseñarse y probarse estos aspectos; no basta con reutilizar `.env`.

Prueba de permisos: una cuenta `usuario` no puede acceder a `/admin/activity`; una cuenta `administrador` sí. Ambas gestionan solo sus propias tareas. La API nunca recibe roles confiables desde el cuerpo de una solicitud.

37. Resolución de problemas

Antes de modificar código anota: comando, directorio, versión, mensaje completo y resultado esperado. Repite una comprobación pequeña para aislar la capa que falla.

Síntoma	Causa posible	Solución y comprobación
flutter command not found	PATH incompleto o terminal antigua	Añadir `flutter/bin`, abrir terminal nueva, `where.exe flutter`
Android sdkmanager not found	Falta Command-line Tools o SDK equivocado	SDK Manager, instalar herramientas, revisar ruta, repetir doctor
Android licenses not accepted	Licencias pendientes	`flutter doctor --android-licenses`; leer y aceptar las necesarias
No devices found	Emulador apagado, cable o controlador	Iniciar AVD o revisar USB; repetir `flutter devices`
ADB device unauthorized	Autorización RSA pendiente	Desbloquear celular y aceptar la computadora; reconectar
INSTALL_FAILED_USER_RESTRICTED	Instalación USB restringida	Atender aviso, configuración del fabricante o política del dispositivo
Puerto Docker ocupado	Otro proceso publica el mismo puerto	Identificarlo y detener solo el proceso conocido o cambiar el puerto
Docker Desktop no inicia	WSL/virtualización/compatibilidad	Revisar virtualización, `wsl --status` y diagnóstico Desktop
PostgreSQL connection refused	Host o puerto equivocado, proceso caído	`docker compose ps`; interno `postgres:5432`, externo `localhost:5432`
MongoDB connection refused	Servicio inaccesible	Logs y puerto; interno `mongo`, externo `localhost`
Error de contraseña en DB	`.env` distinto del volumen existente	Usar credencial inicial o rotarla dentro de la DB; no borrar datos por impulso
Keycloak no inicia	DB no disponible, configuración o memoria	Logs de `keycloak` y `keycloak-db`; comprobar disponibilidad y recursos
Realm sin cambios tras editar JSON	Importación solo inicial	Aplicar cambios desde consola o importar en un entorno nuevo de prueba
401 Unauthorized	Token ausente, vencido o claims inválidos	Renovar; comparar issuer/audience; usar access token y reloj correcto
403 Forbidden	Rol insuficiente	Asignar rol y obtener un token nuevo
CORS en navegador	Origen no permitido	Añadir origen exacto a CORS_ORIGINS y reiniciar backend

Síntoma	Causa posible	Solución y comprobación
Flutter no conecta a localhost	localhost apunta al teléfono	Configurar ADB reverse o red LAN consistente; capítulo 32
Login no vuelve a Flutter	Redirect/manifest no coincide	Revisar esquema, URI exacta y recompilar Android
HTTP bloqueado en Android	Se ejecuta release con URLs HTTP	Usar debug local o configurar HTTPS para distribución
Git push rejected	Remoto avanzó o faltan permisos	Fetch, revisar historial, integrar cambios y reintentar
Git branch no encontrada	Nombre incorrecto, mayúsculas o rama solo remota	`git branch -a`, fetch y crear seguimiento si corresponde

Comandos de diagnóstico

```
Get-NetTCPConnection -LocalPort 8000 -ErrorAction SilentlyContinue
```

```
docker compose logs --tail 80 keycloak
```

```
docker compose logs --tail 80 backend
```

```
wsl --status
```

```
git fetch origin
```

```
git branch -a
```

Si tu rama existe solo en remoto:

```
git switch --track origin/feature/api
```

Si un push falla por cambios ajenos, revisa `git status` y el historial. Con trabajo guardado, integra conscientemente mediante merge o rebase según el acuerdo del equipo. No uses `push --force` para ocultar un conflicto.

CORS protege llamadas desde navegadores; no es un firewall ni explica por sí solo una conexión rechazada en Android nativo. Un 401 demuestra que hubo comunicación HTTP; una conexión rechazada indica que no se llegó al servicio.

Certificados TLS de una red administrada

Si npm, pip o Gradle muestran `UNABLE_TO_VERIFY_LEAF_SIGNATURE`, `CERTIFICATE_VERIFY_FAILED` o `PKIX path building failed`, puede existir una CA de la red que Windows reconoce pero el runtime no. Confirma con el administrador qué certificado debe confiarse. No desactives TLS ni uses registros HTTP.

Para Node moderno en Windows, en la sesión actual:

```
$env:NODE_USE_SYSTEM_CA = '1'
```

Para el build Docker se incluye una ruta opcional: exporta únicamente los certificados públicos ya confiados por Windows y pásalos al build como un montaje temporal:

```
powershell -File scripts/export_windows_ca.ps1
```

```
docker compose --env-file .env -f infrastructure/docker-compose.yml -f infrastructure/docker-compose.ca.yml up -d --build
```

El archivo `.local/windows-ca.pem` está excluido de Git y no se copia a la imagen. En una red normal basta con el Compose principal. Para Gradle/JDK en Windows puedes probar el almacén del sistema en esa terminal:

```
$env:JAVA_TOOL_OPTIONS = '-Djavax.net.ssl.trustStoreType=Windows-ROOT -Djavax.net.ssl.trustStore=NONE'
```

Después repite la compilación. Si tu JDK no admite ese almacén, solicita una CA verificada y configura un truststore específico para ese proceso. Cierra la terminal para descartar estas variables temporales.

Referencias: CA en Docker, certificados pip, Node y certificados del sistema, truststores Java.

38. Evidencias de aprendizaje

Cada archivo E01–E24 de evidence contiene objetivo, comando o acción, resultado esperado, espacio para captura y conclusión. Completa el resultado real; no marques éxito sin ejecutar la comprobación.

Evidencia	Módulo
E01	Git instalado e historial
E02	Flutter doctor
E03	Emulador o Android físico
E04	Primera aplicación Flutter
E05	API funcionando
E06	Swagger / OpenAPI
E07	Postman
E08	PostgreSQL
E09	MongoDB
E10	Realm y clientes Keycloak
E11	Docker
E12	Docker Compose
E13	GitHub, PR e Issue
E14	Flutter consumiendo API
E15	Hardware, PATH y entorno
E16	VS Code y extensiones
E17	Dart
E18	Android Studio y SDK
E19	Python / FastAPI, si se eligió
E20	Node.js / Express, si se eligió
E21	Autenticación y permisos
E22	Dockerfile y build
E23	Variables y exclusión de secretos
E24	Pruebas finales y versiones

Usa nombres de captura como `E14-crear-tarea.png`. Antes de incluirlos, oculta tokens, contraseñas y datos personales. Puedes conservar material sensible temporal en `docs/evidence/private`, excluido de Git, y publicar solo versiones revisadas.

Una conclusión útil relaciona resultado con concepto: “La tarea permaneció al reiniciar porque el volumen conserva PostgreSQL”. Evita conclusiones genéricas como “todo salió bien”. Si eliges un solo backend, marca el otro módulo como “alternativa no elegida”.

39. Actividad final: gestión de tareas

Producto esperado

Una aplicación Android en Flutter con login, lista, creación, edición, eliminación, perfil y cierre de sesión. El backend elegido ofrece:

Método	Ruta	Acceso
GET	`/`	Público, información básica
GET	`/health`	Público, salud de bases
GET	`/me`	Token válido
GET	`/tasks`	Usuario/administrador, tareas propias
POST	`/tasks`	Usuario/administrador
PUT	`/tasks/{id}`	Propietario
DELETE	`/tasks/{id}`	Propietario
GET	`/categories`	Usuario/administrador
GET / POST	`/usuarios`	Administrador, ejercicio de usuarios
GET	`/admin/activity`	Administrador

PostgreSQL contiene `users`, `tasks` y `categories`; `usuarios` es una tabla educativa independiente. MongoDB contiene `activity` y `logs`. Keycloak contiene cuentas y roles, sin guardar contraseñas en la tabla `users` de aplicación.

Procedimiento de entrega

1. Configura `.env` y levanta la alternativa elegida con Compose.
2. Crea dos cuentas `usuario`, una `administrador` y una sin roles de aplicación.
3. Prepara ADB reverse y ejecuta Flutter.
4. Inicia sesión como alumno, crea “Preparar exposición”, edítala, márcala completada y recarga.
5. Comprueba la fila en PostgreSQL y eventos en MongoDB.
6. Inicia sesión con otra cuenta y demuestra que no ve ni modifica la tarea anterior.
7. Prueba 401 sin token y 403 con la cuenta sin roles.
8. Reinicia el backend conservando volúmenes y verifica persistencia.
9. Elimina la tarea, comprueba el evento y cierra sesión.
10. Publica código y evidencias revisadas en GitHub mediante PR.

```
docker compose up -d
```

Flutter se ejecuta en Android desde el SDK; Compose ejecuta **todo el backend**, no un emulador Android.

Flujo que debe demostrarse

Usuario → Flutter → Keycloak → JWT → API → PostgreSQL/MongoDB → respuesta → Flutter.

Límites explicados

La escritura de la tarea y la auditoría no son atómicas entre ambas bases. Si MongoDB falla después del commit SQL, se devuelve éxito con `audit_recorded: false` y Flutter avisa. Esto evita informar un fracaso de creación cuando la tarea sí existe. Una entrega que requiera auditoría garantizada deberá añadir y probar un patrón outbox o una cola con reintentos.

La lista es pequeña y no implementa paginación. La autenticación utiliza navegador y almacenamiento seguro; las pruebas unitarias no sustituyen la demostración en dispositivo. Las credenciales y capturas reales las aporta el estudiante.

Evaluación propuesta

Área	Puntos
Instalación y conceptos	20
API y persistencia	25
Autenticación y permisos	20
Flutter y manejo de errores	20
GitHub, evidencias y explicación	15

40. Checklist final

- [ ] Git instalado.
- [ ] GitHub configurado.
- [ ] Flutter instalado.
- [ ] Dart reconocido.
- [ ] Flutter doctor correcto para Android.
- [ ] Android SDK instalado y licencias aceptadas.
- [ ] Emulador funcionando o teléfono físico verificado.
- [ ] Postman instalado.
- [ ] Backend elegido funcionando.
- [ ] Swagger funcionando.
- [ ] PostgreSQL conectado.
- [ ] MongoDB conectado.
- [ ] Keycloak funcionando.
- [ ] Docker funcionando.
- [ ] Docker Compose funcionando.
- [ ] Flutter consume la API.
- [ ] Autenticación funcionando.
- [ ] Tokens inválidos y roles insuficientes rechazados.
- [ ] Usuarios aislados por propietario.
- [ ] Datos persistentes tras reinicio.
- [ ] `.env` y tokens excluidos de Git.
- [ ] Pruebas y limitaciones registradas.
- [ ] Proyecto almacenado en GitHub.
- [ ] Portada, versiones y evidencias completas.

Marca cada casilla después de su comprobación. El material generado no constituye evidencia de que todas las herramientas estén configuradas en una computadora nueva.

41. Glosario

Concepto	Definición
API	Interfaz para que programas intercambien operaciones y datos.
REST	Estilo de arquitectura basado en recursos y una interfaz uniforme.
HTTP	Protocolo de solicitudes y respuestas.
HTTPS	HTTP protegido mediante TLS.
JSON	Formato textual de intercambio de datos.
JWT	Token con claims y firma verificable.
OAuth 2.0	Marco de autorización delegada.
OpenID Connect	Protocolo de identidad sobre OAuth 2.0.
PKCE	Prueba que vincula el código de autorización con quien inició el flujo.
SDK	Conjunto de herramientas para desarrollar para una plataforma.
Framework	Estructura y herramientas para construir aplicaciones.
Runtime	Entorno que ejecuta un programa.
Endpoint	Dirección y operación accesible de una API.
Backend	Lógica y datos del lado servidor.
Frontend	Interfaz con la que interactúa el usuario.
ORM	Herramienta que relaciona objetos con datos relacionales.
SQL	Lenguaje de definición y consulta relacional.
NoSQL	Familia de modelos de datos no limitados al relacional.
BSON	Representación binaria de documentos usada por MongoDB.
Contenedor	Proceso aislado creado desde una imagen.
Imagen	Plantilla de un entorno de ejecución.
Volumen	Almacenamiento persistente administrado fuera del contenedor.
Repositorio	Archivos y su historial de cambios.
Commit	Registro identificado de cambios.
Branch	Línea de trabajo en Git.
Merge	Integración de historiales.
Pull Request	Propuesta de cambios para revisión e integración.

Concepto	Definición
Token	Credencial o dato representativo utilizado por un protocolo.
Realm	Dominio de identidades y configuración en Keycloak.
Client	Aplicación registrada en el proveedor de identidad.
Role	Conjunto o categoría de permisos.
Middleware	Componente que interviene en el procesamiento de solicitudes.
IAM	Gestión de identidades y acceso.
Claim	Afirmación incluida en un token.
JWKS	Conjunto de claves públicas para verificar firmas.
Issuer	Emisor de un token.
Audience	Destinatario previsto de un token.
CORS	Política que regula solicitudes de origen cruzado en navegadores.
PATH	Lista de directorios para buscar ejecutables.
AVD	Configuración de dispositivo Android virtual.
ADB	Herramienta de comunicación y depuración Android.
Hot Reload	Aplicación de cambios de código preservando parte del estado.
Timeout	Límite de espera de una operación.
CRUD	Crear, leer, actualizar y eliminar.
Healthcheck	Comprobación periódica de disponibilidad de un servicio.
Transacción	Grupo de operaciones con garantías de consistencia.
Clave primaria	Identificador único de una fila.
Clave foránea	Referencia que relaciona filas entre tablas.
Idempotencia	Propiedad por la que repetir una operación no añade efectos distintos.